# Homework - 5

**Repository:** https://github.com/Ani2508/data260-3209

**Domain:** Clinical Trial Listing

## Section 0 - Personal Configuration

| Value | Result |
|---|---|
| SID4 | 3209 |
| PORT_BASE | 8509 (8000 + 3209 mod 900) |
| PREFIX | s3209 |
| SEED | 3209 |
| VERIFY_SEED | 263209 |
| DOMAIN_ID | 1 - Clinical trial listings |

### Environment

| | |
|---|---|
| Hardware | Apple MacBook Pro, Apple M5 Pro chip |
| RAM | 48 GB |
| OS | macOS |
| Local model | llama3.1:8b (Ollama) |

## Part 1 - FastAPI + MySQL Backend

### I. Database

Sponsor is the related entity (like author) and Trial is the primary entity. One sponsor runs many trials.

- **sponsors:** id (PK), name, country, contact_email (unique), created_at, updated_at
- **trials:** id (PK), trial_title, nct_id (unique, NCT + 8 digits), enrollment_slots (default 0), status, sponsor_id (FK to sponsors.id), created_at, updated_at
- Passwords are stored only as bcrypt hashes
- A sponsor with trials cannot be deleted (ON DELETE RESTRICT and the API returns 409)

**Code Snippet:**

```python

class Sponsor(Base):
    """Related entity (the "author" role): one sponsor runs many trials."""
    __tablename__ = "sponsors"
    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(255), nullable=False)                 # primary text field
    country = Column(String(100), nullable=False)              # secondary text field
    contact_email = Column(String(255), nullable=False, unique=True)  # unique field
    created_at = Column(DateTime, nullable=False, server_default=func.now())
    updated_at = Column(DateTime, nullable=False, server_default=func.now(),
                        onupdate=func.now())

    # No cascade: a sponsor that still has trials cannot be deleted (RESTRICT).
    trials = relationship("Trial", back_populates="sponsor", passive_deletes="all")

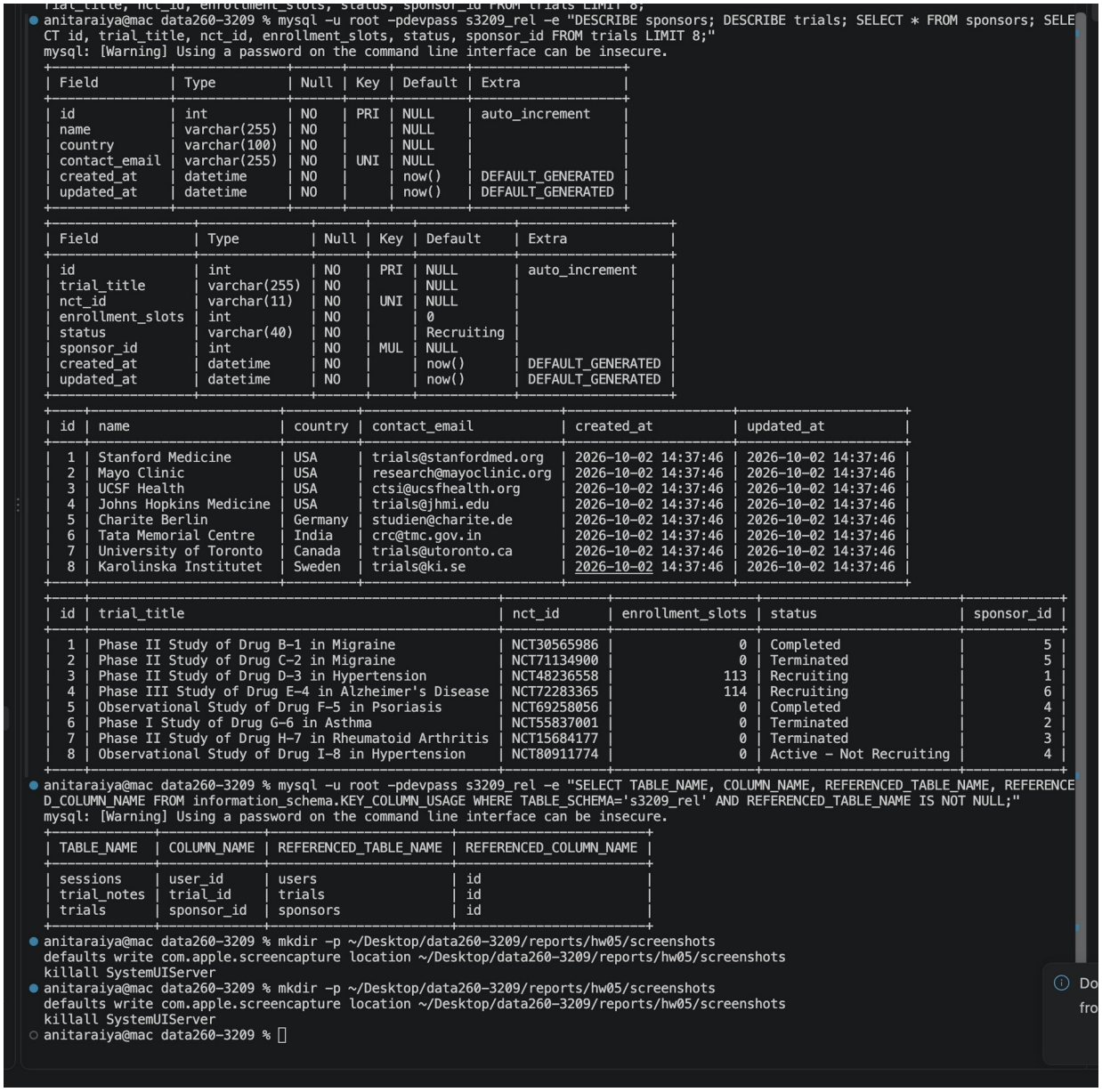

class Trial(Base):
    """Primary domain entity: one clinical trial listing."""
    __tablename__ = "trials"
    id = Column(Integer, primary_key=True, autoincrement=True)
    trial_title = Column(String(255), nullable=False)          # primary field
    nct_id = Column(String(11), nullable=False, unique=True)    # unique code, e.g. NCT01234567
    enrollment_slots = Column(Integer, nullable=False, default=0,
                              server_default="0")              # numeric field, default 0
    status = Column(String(40), nullable=False, default="Recruiting",
                    server_default="Recruiting")
    sponsor_id = Column(Integer, ForeignKey("sponsors.id", ondelete="RESTRICT"),
                        nullable=False, index=True)            # foreign key
    created_at = Column(DateTime, nullable=False, server_default=func.now())
    updated_at = Column(DateTime, nullable=False, server_default=func.now(),
                        onupdate=func.now())

    sponsor = relationship("Sponsor", back_populates="trials")
    notes = relationship("TrialNote", cascade="all, delete-orphan")


```

Sponsor and Trial tables

**Output Snippet:**




MySQL tables foreign key and sample rows

## II. API Implementation

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /api/sponsors | Add a sponsor |
| GET | /api/sponsors?<br>page=1&page_size=5 | List sponsors (paged) |
| GET | /api/sponsors/{id} | Get one sponsor |
| PUT | /api/sponsors/{id} | Update a sponsor |
| DELETE | /api/sponsors/{id} | Delete a sponsor |
| POST | /api/trials | Add a trial |
| GET | /api/trials? | List trials (paged) |

| | | |
|---|---|---|
| | page=1&page_size=5 | |
| GET | /api/trials/{id} | Get one trial |
| PUT | /api/trials/{id} | Update a trial |
| DELETE | /api/trials/{id} | Delete a trial |
| GET | /api/sponsors/{id}/trials | All trials of one sponsor |

Pydantic validates every request. Status codes: 201 created, 200 OK, 404 not found, 422 invalid input (email or NCT format, unknown sponsor_id) and 409 conflict (duplicate email or NCT ID, sponsor still has trials).

**Code Snippet:**

```python
# ---------------- Sponsor (related entity) ----------------
class SponsorIn(BaseModel):
    name: str = Field(min_length=1, max_length=255, examples=["Stanford Medicine"])
    country: str = Field(min_length=1, max_length=100, examples=["USA"])
    contact_email: str = Field(max_length=255, examples=["trials@stanford.edu"])

    _strip = field_validator("name", "country")(_not_blank)

    @field_validator("contact_email")
    @classmethod
    def check_email(cls, value: str) -> str:
        value = value.strip().lower()
        if not EMAIL_RE.match(value):
            raise ValueError("contact_email must look like name@domain.com")
        return value


class SponsorOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    country: str
    contact_email: str
    created_at: datetime
    updated_at: datetime


# ---------------- Trial (primary entity) ----------------
class TrialIn(BaseModel):
    trial_title: str = Field(min_length=1, max_length=255,
                             examples=["Phase II Study of Drug X in Type 2 Diabetes"])
    nct_id: str = Field(examples=["NCT01234567"])
    enrollment_slots: int = Field(default=0, ge=0, le=100000)
    status: Status = "Recruiting"
    sponsor_id: int = Field(gt=0)

    _strip = field_validator("trial_title")(_not_blank)

    @field_validator("nct_id")
    @classmethod
    def check_nct(cls, value: str) -> str:
        value = value.strip().upper()
        if not NCT_RE.match(value):
            raise ValueError("nct_id must be 'NCT' followed by 8 digits, e.g. NCT01234567")
        return value
```

Request validation

```python

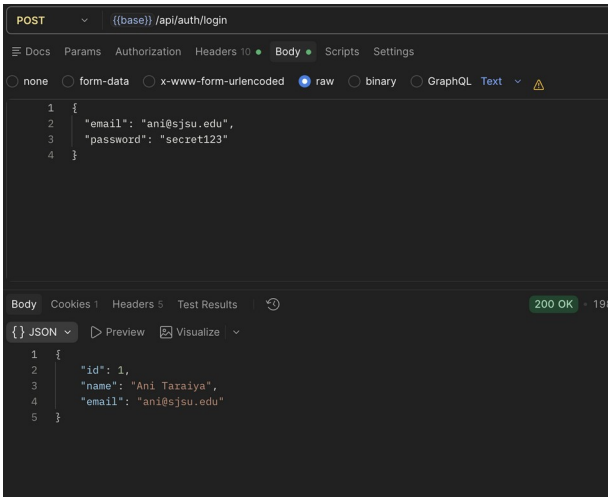
@router.delete("/{sponsor_id}")
def delete_sponsor(sponsor_id: int, db: Session = Depends(get_db)):
    sponsor = get_sponsor_or_404(db, sponsor_id)
    linked = db.query(func.count(Trial.id)).filter(Trial.sponsor_id == sponsor_id).scalar()
    if linked:
        raise HTTPException(
            status_code=409,
            detail=f"Sponsor {sponsor_id} still has {linked} trial(s); "
                   "delete or reassign them before deleting the sponsor")
    db.delete(sponsor)
    db.commit()
    return {"message": "Sponsor deleted", "id": sponsor_id}


# Relationship query: all trials run by one sponsor
@router.get("/{sponsor_id}/trials", response_model=list[TrialOut])
def trials_for_sponsor(sponsor_id: int, db: Session = Depends(get_db)):
    get_sponsor_or_404(db, sponsor_id)
    rows = (db.query(Trial).options(joinedload(Trial.sponsor))
            .filter(Trial.sponsor_id == sponsor_id).order_by(Trial.id).all())
    return [trial_out(t) for t in rows]

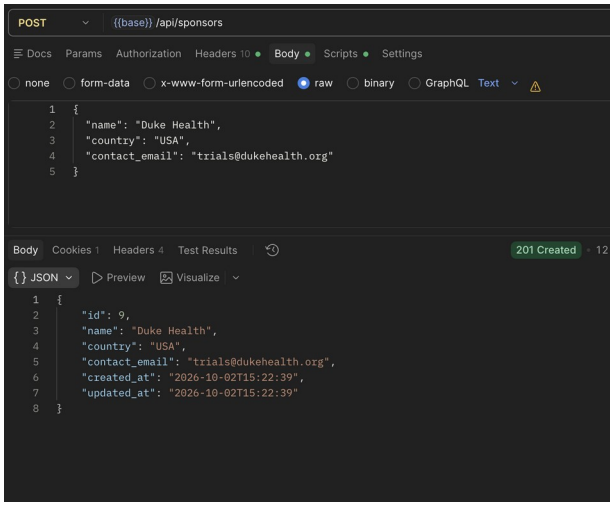
```

Delete rule and relationship endpoint

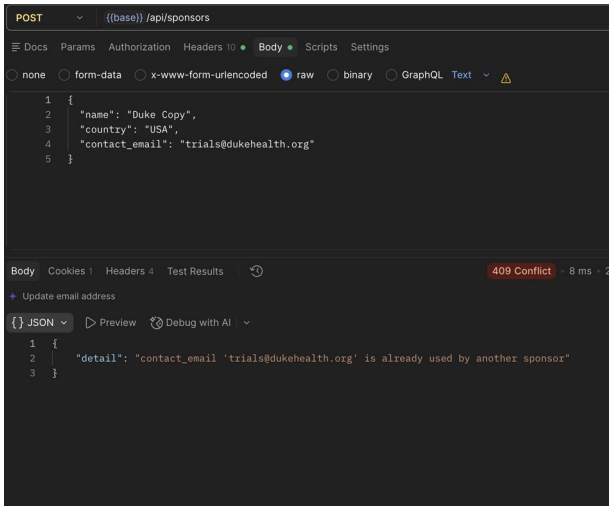
**Output Snippet (Postman):**



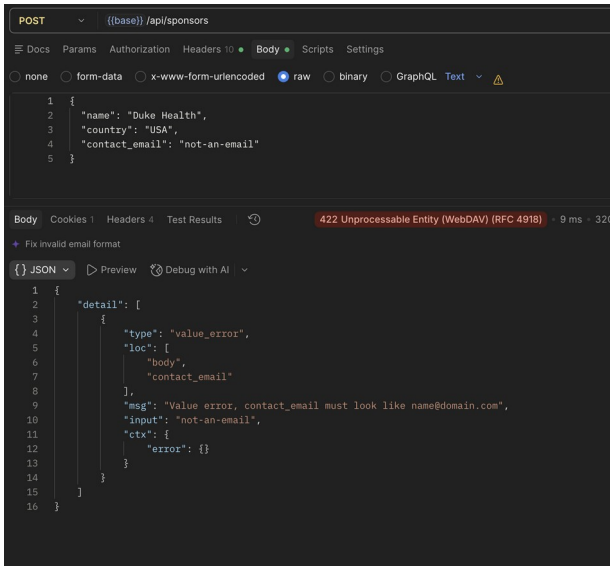

Login (200)



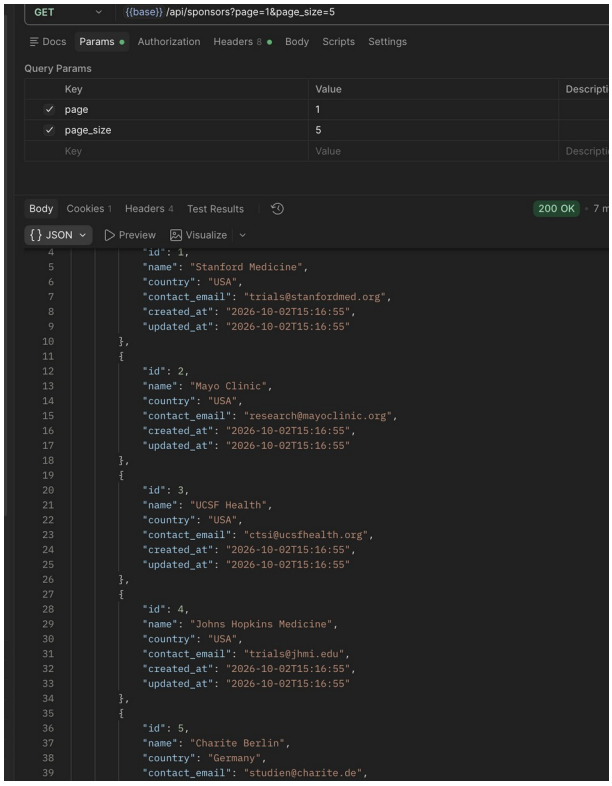

Create sponsor (201)



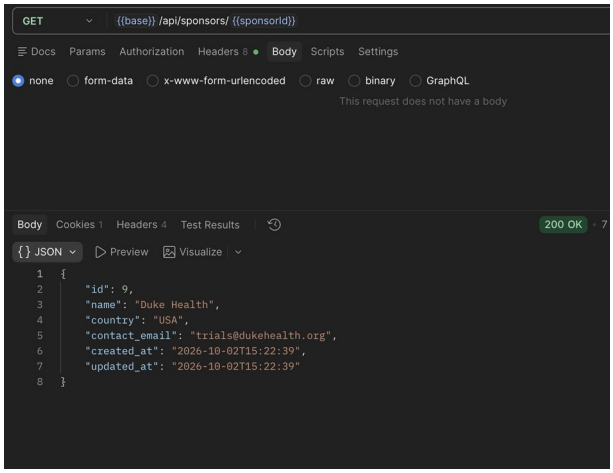

Duplicate contact email (409)



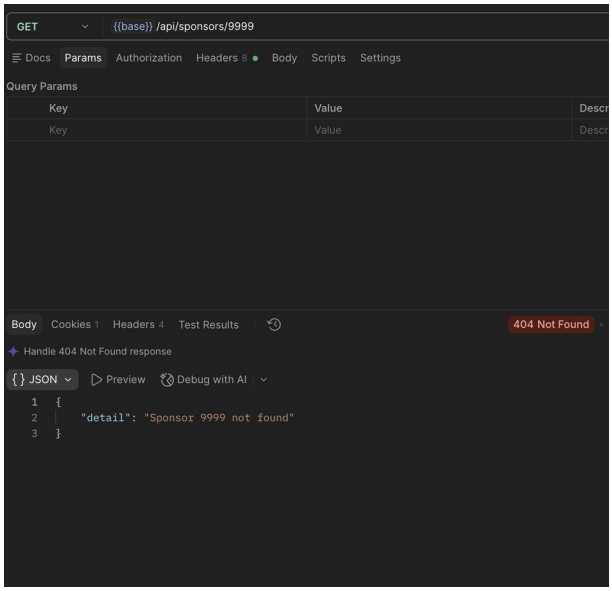

Invalid email format (422)



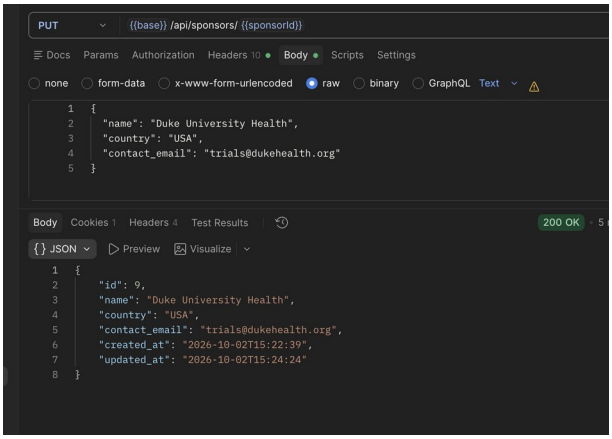

List sponsors with pagination (200)



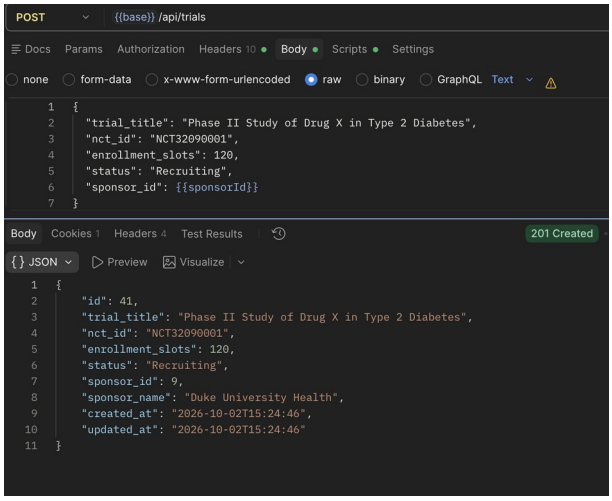

Get one sponsor (200)



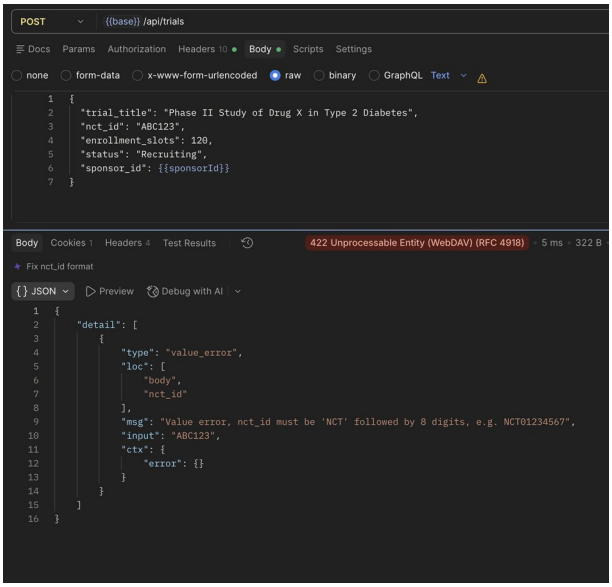

Sponsor not found (404)



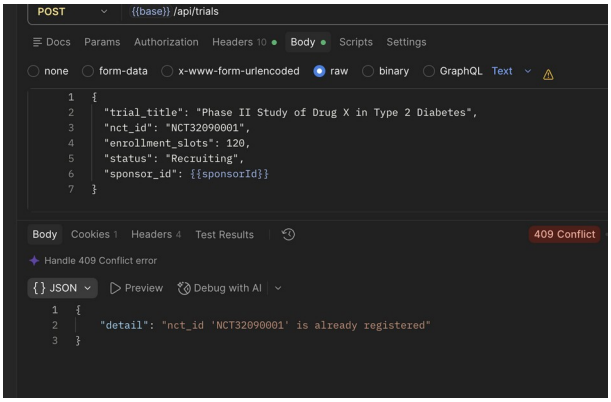

Update sponsor (200)



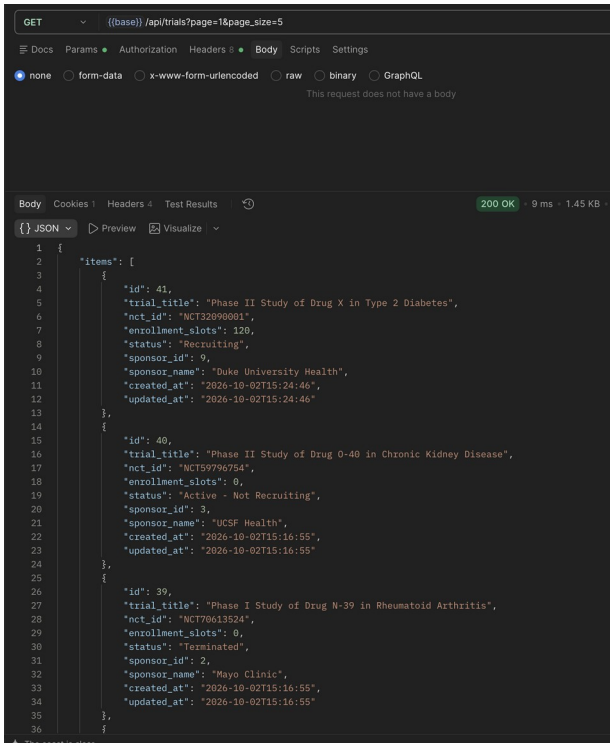

Create trial (201)



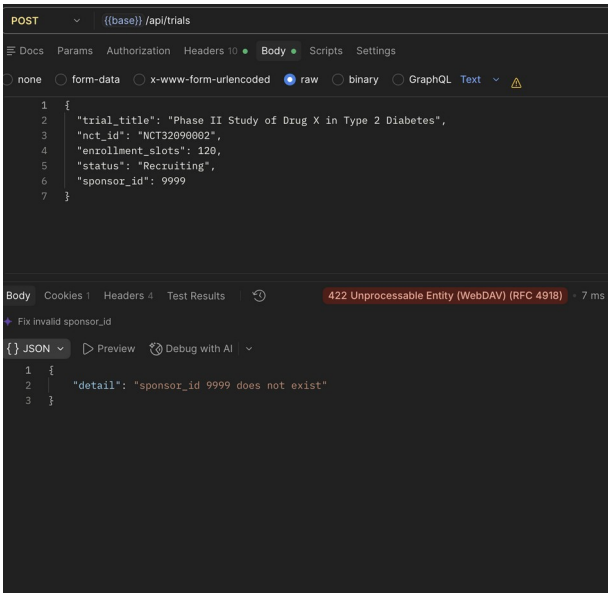

Invalid NCT ID format (422)



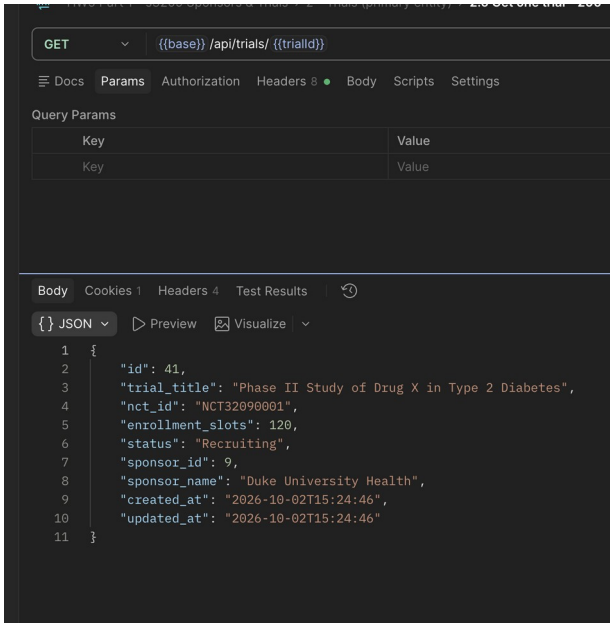

Duplicate NCT ID (409)



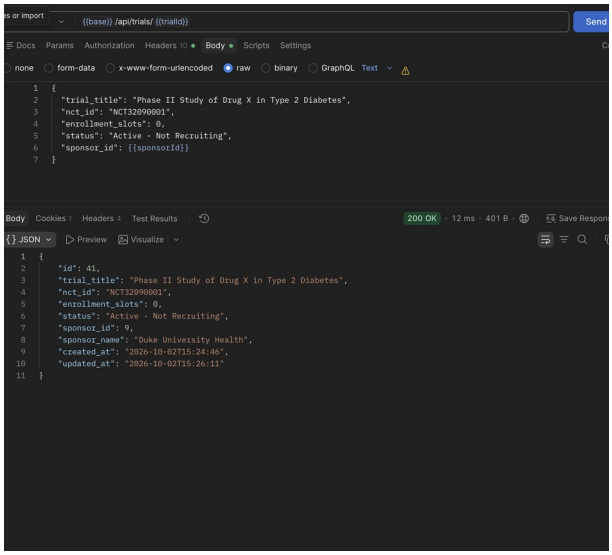

Unknown sponsor_id (422)



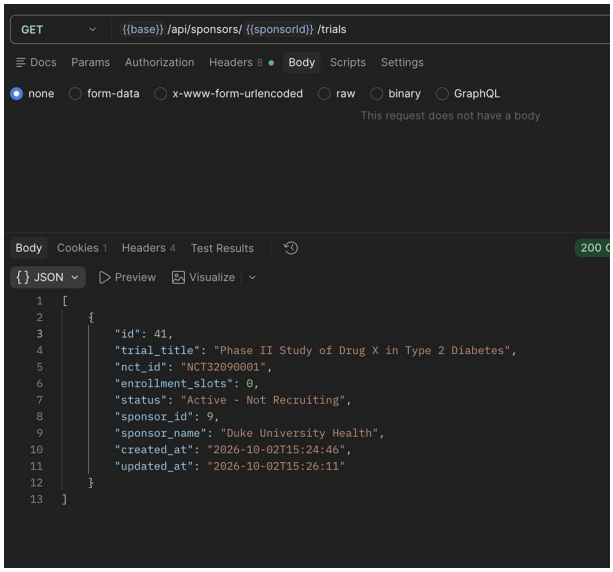

List trials with pagination (200)



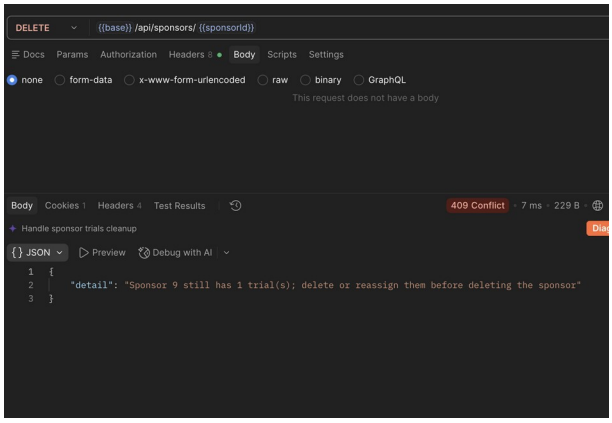

Get one trial (200)



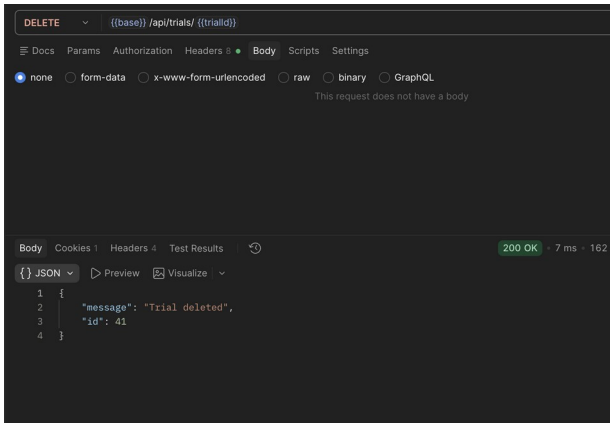

Update trial (200)



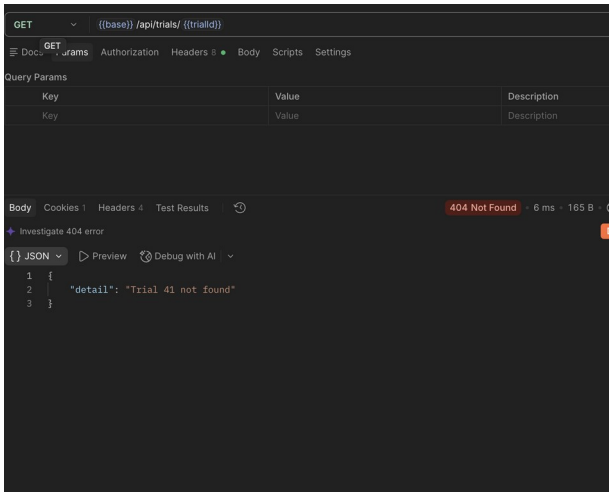

Trials of one sponsor (200)



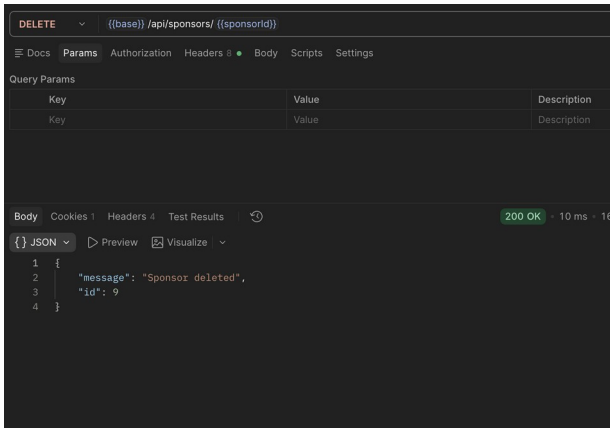

Delete sponsor with trials (409)




Delete trial (200)




Get deleted trial (404)




Delete sponsor with no trials (200)

## III. Redux Client

### 1. Redux Setup

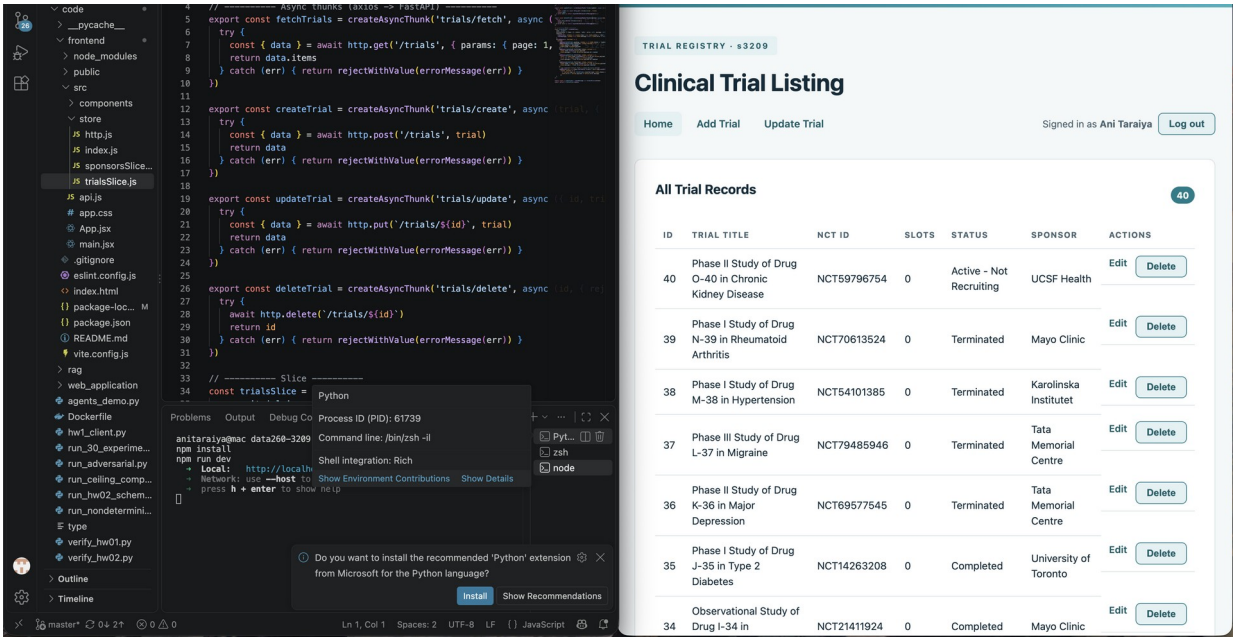
store/index.js creates the store with configureStore. trialsSlice.js holds the trial records with reducers for fetching, adding, updating and deleting.

### 2. Async Thunks with Axios

fetchTrials (GET), createTrial (POST), updateTrial (PUT /{id}) and deleteTrial (DELETE /{id}) call the FastAPI endpoints with axios. Errors are returned with rejectWithValue and shown on the page.

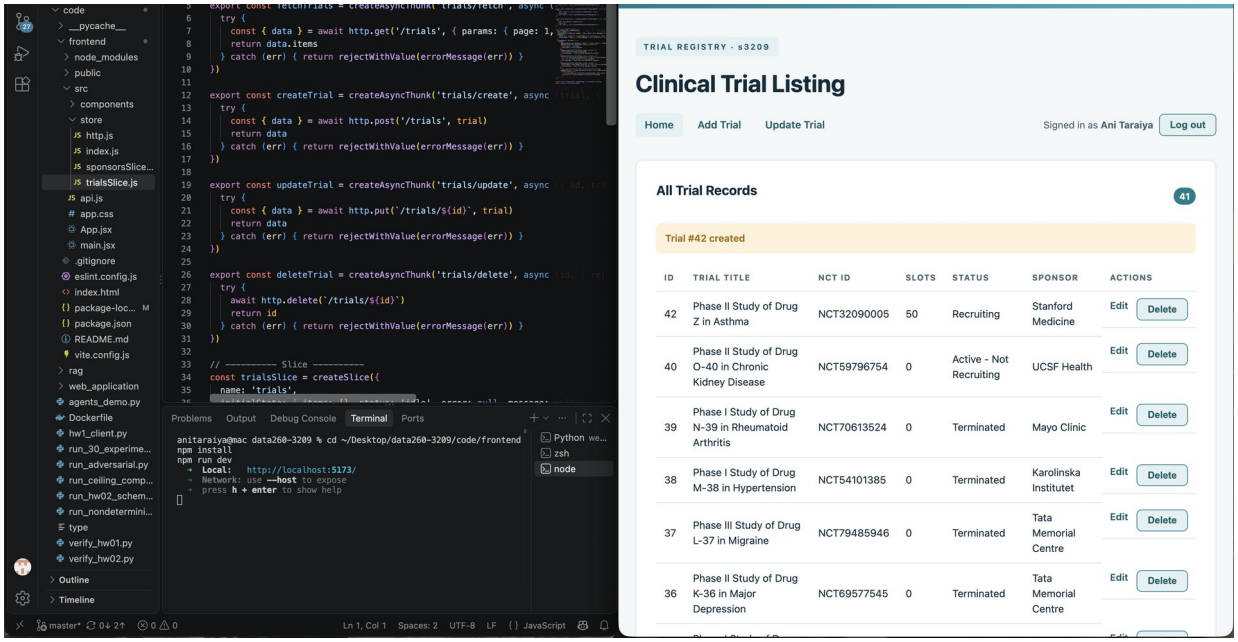
### 3. Home Screen

Home reads the list with useSelector, so it updates after every create, update or delete.

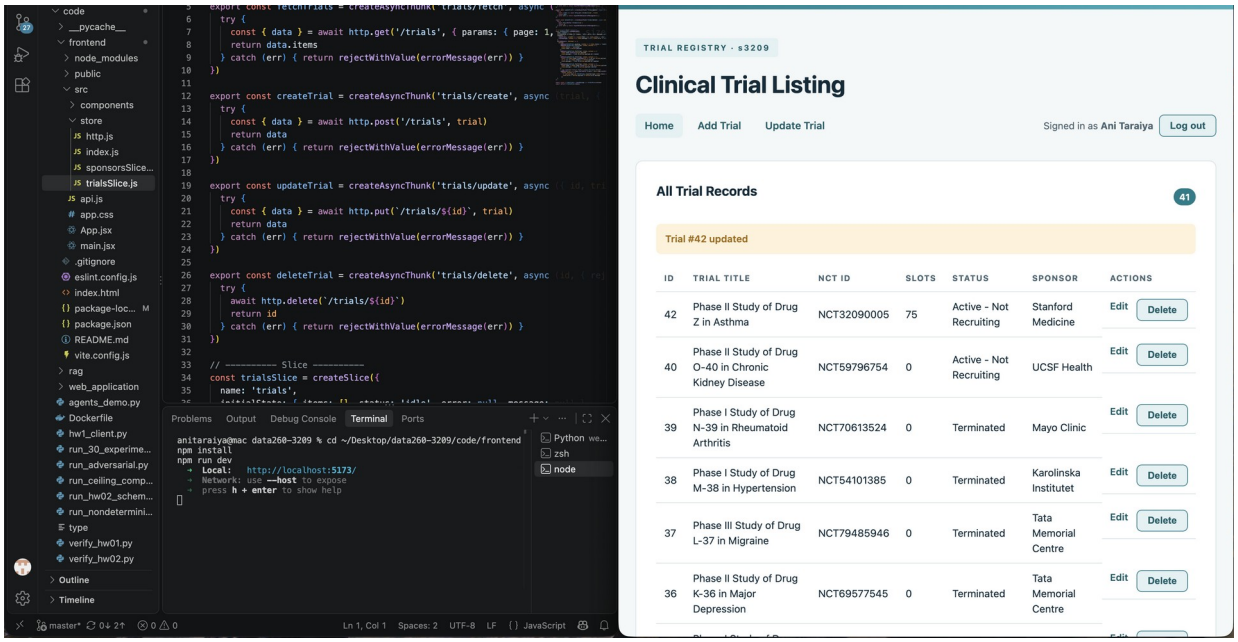



Home - fetchTrials

## 4. Create / Update Screens

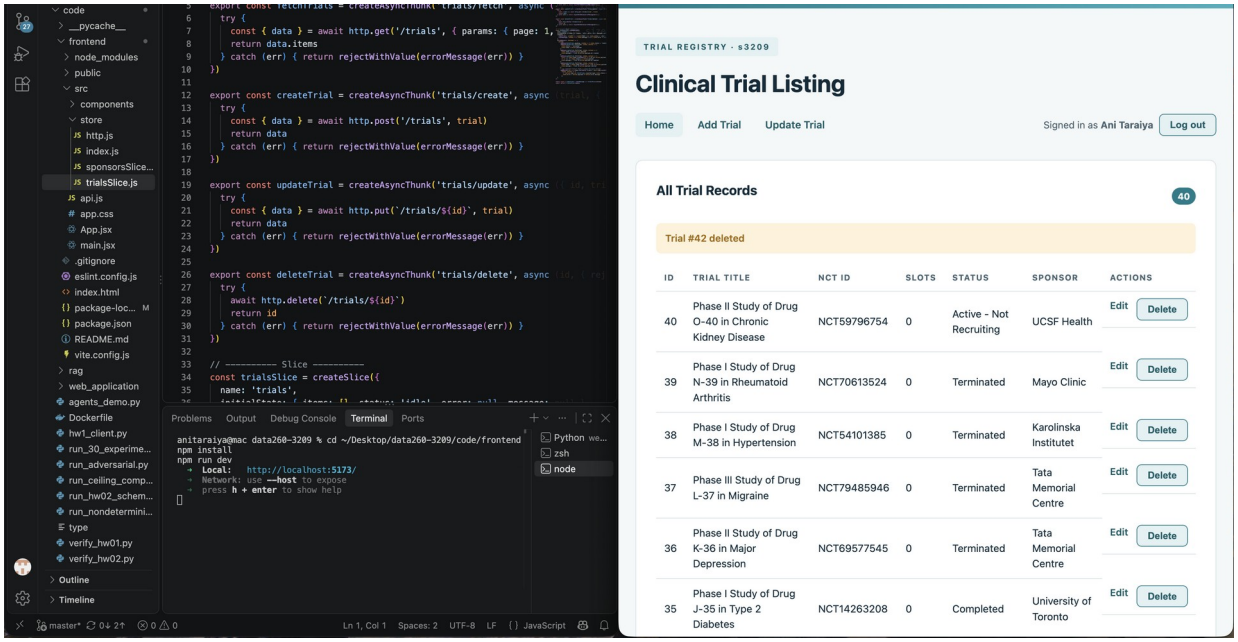



Create - createTrial (trial #42 added)

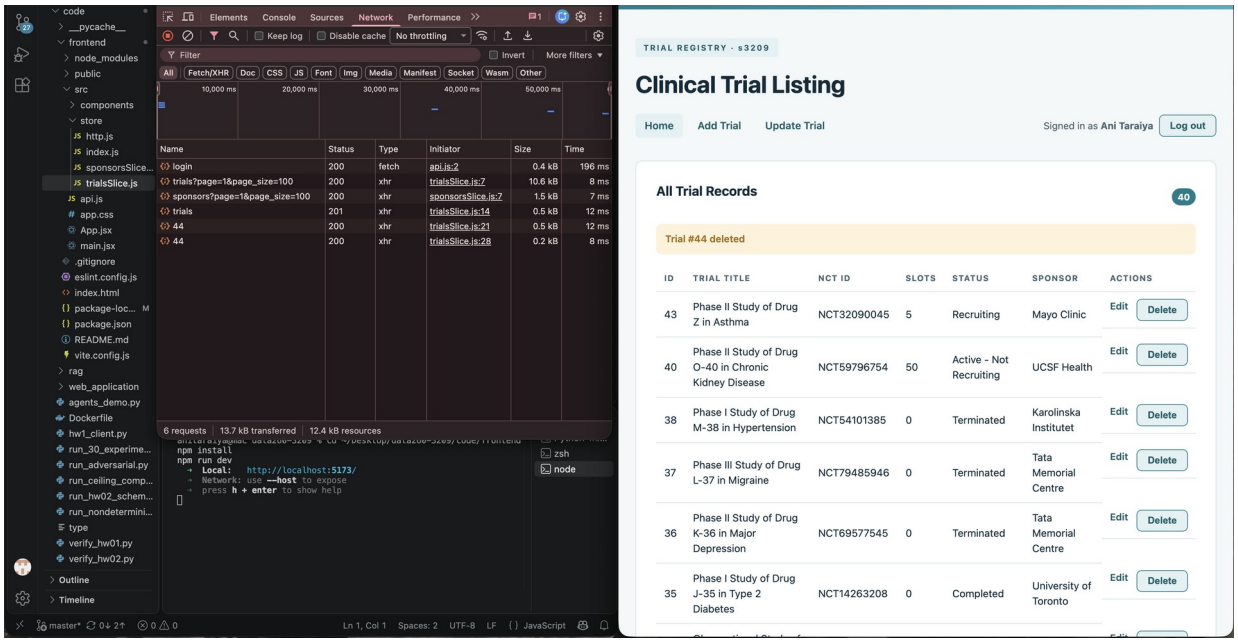



Update - updateTrial (trial #42 changed)

### 5. Delete Functionality




Delete - deleteTrial (trial #42 removed)



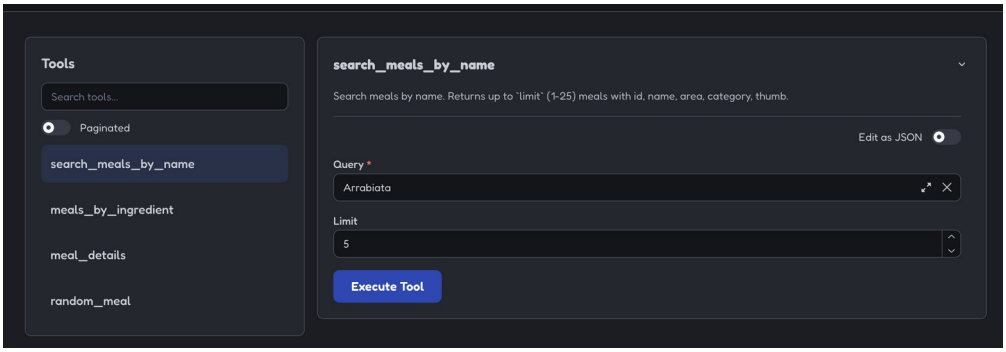

Network tab - POST 201, PUT 200 and DELETE 200

# Part 2 - MCP Tool Servers

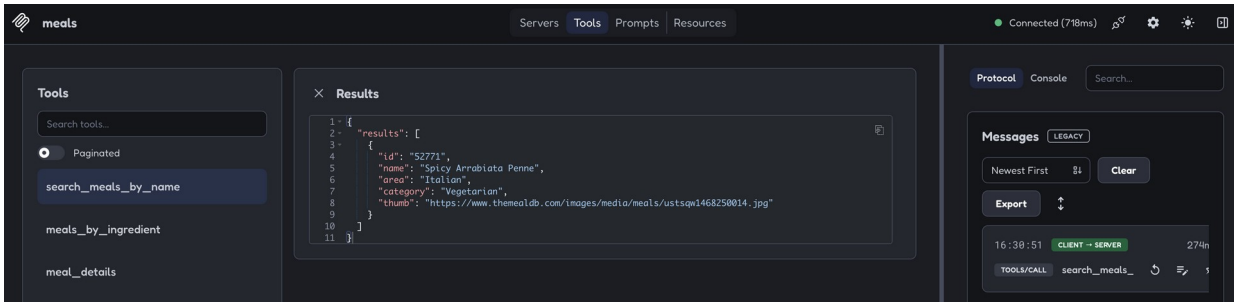
## A. TheMealDB Tutorial Server

MCP server named meals (meals_server.py) using FastMCP over STDIO and TheMealDB test key 1.

## 6. search_meals_by_name(query, limit=5)

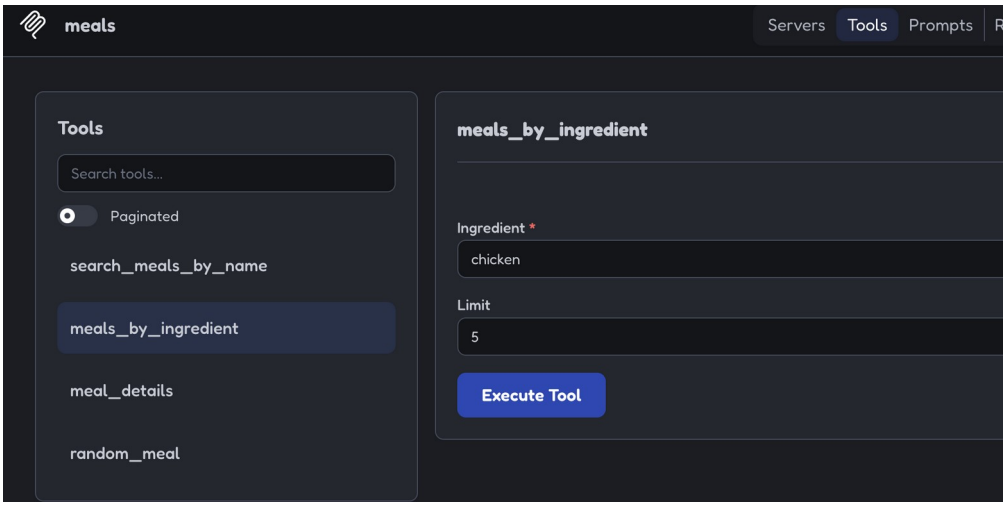



Input

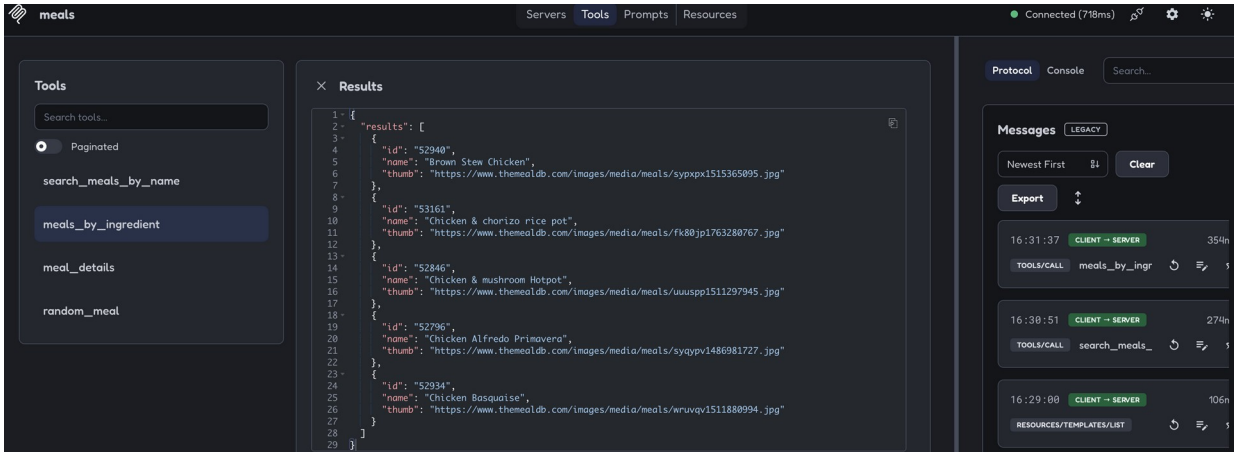



Output

## 7. meals_by_ingredient(ingredient, limit=12)

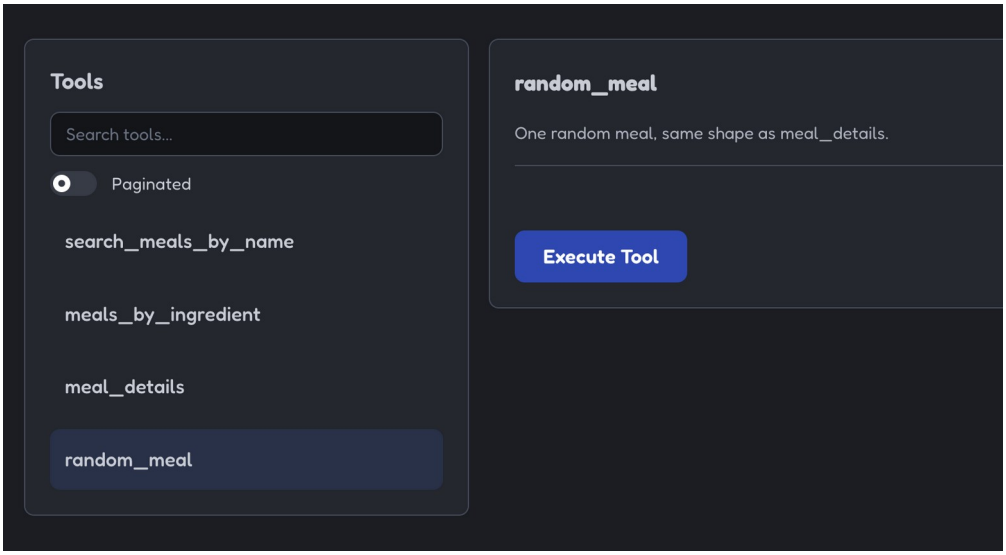



Input

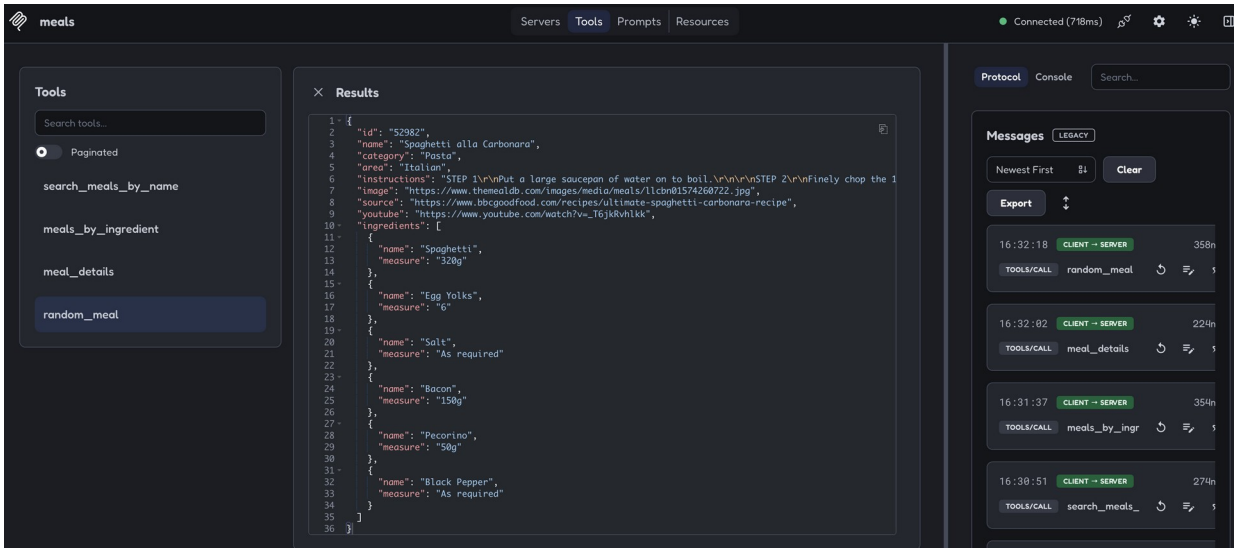



Output

## 8. random_meal()

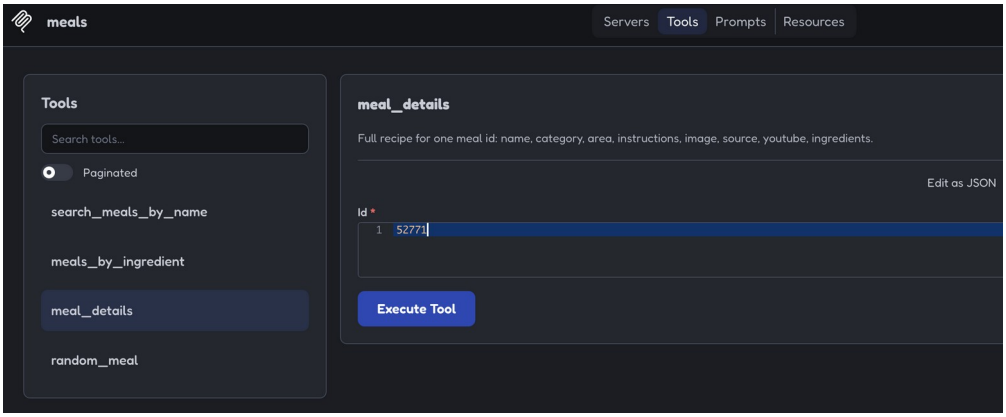



Input

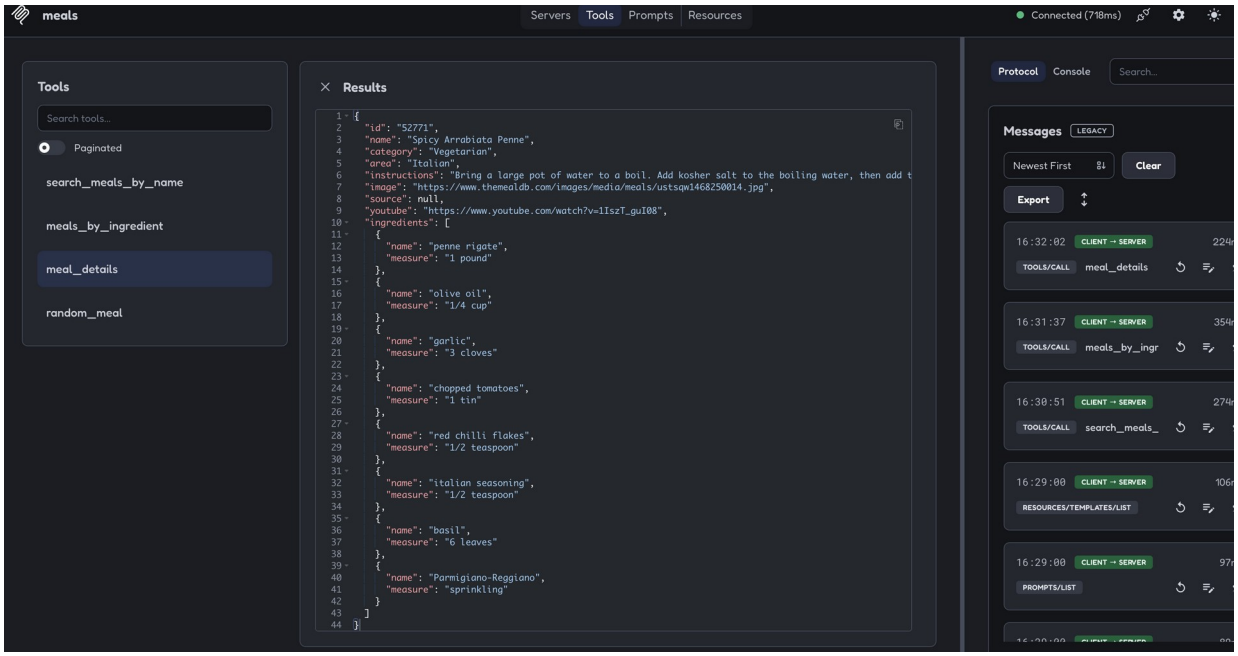



Output

## 9. meal_details(id)




Input




Output

## 10-11. Environment setup

Installed the SDK with pip install "mcp[cli]" and wrote one file, meals_server.py. Logs go to stderr only.

```python
def _get(endpoint: str, params: dict | None = None) -> list[dict] | None:
    """Call TheMealDB and return the "meals" list (None when the API says no results).

    Network / HTTP / JSON problems raise RuntimeError with a short message, which
    the MCP client (Inspector) shows as a clean tool error.
    """
    url = f"{API_BASE}/{endpoint}"
    log.info("GET %s params=%s", url, params)
    try:
        resp = httpx.get(url, params=params, timeout=TIMEOUT_S)
        resp.raise_for_status()
        payload = resp.json()
    except httpx.TimeoutException as exc:
        raise RuntimeError(f"TheMealDB timed out after {TIMEOUT_S}s") from exc
    except httpx.HTTPStatusError as exc:
        raise RuntimeError(f"TheMealDB returned HTTP {exc.response.status_code}") from exc
    except httpx.HTTPError as exc:
        raise RuntimeError(f"Network error calling TheMealDB: {exc}") from exc
    except ValueError as exc:
        raise RuntimeError("TheMealDB returned invalid JSON") from exc
    return payload.get("meals")
```

API call with timeout and clean errors

```python
@mcp.tool()
def search_meals_by_name(query: str, limit: int = 5) -> dict:
    """Search meals by name. Returns up to `limit` (1–25) meals with id, name, area, category, thumb."""
    if not query.strip():
        raise ValueError("query must not be empty")
    if not 1 <= limit <= 25:
        raise ValueError("limit must be between 1 and 25")
    meals = _get("search.php", {"s": query.strip()})
    if not meals:
        return {"results": [], "message": f"no matches for '{query}'"}
    return {"results": [{"id": m["idMeal"], "name": m["strMeal"], "area": m.get("strArea"),
                         "category": m.get("strCategory"), "thumb": m.get("strMealThumb")}
                        for m in meals[:limit]]}
```

search_meals_by_name tool

### 12-15. Input -> Output shapes

| # | Tool | Input | Output |
|---|---|---|---|
| 12 | search_meals_by_name | query: str, limit: int (1-25) | list of {id, name, area, category, thumb} |
| 13 | meals_by_ingredient | ingredient: str, limit: int | list of {id, name, thumb} |
| 14 | meal_details | id: str \| int | {id, name, category, area, instructions, image, source, youtube, ingredients} |
| 15 | random_meal | none | same as meal_details |

If the API returns "meals": null the tool returns an empty result with "no matches". Network or JSON errors return a clean error to the client.

### 16-17. Run and debug locally with MCP Inspector

Ran mcp dev meals_server.py and tested each tool in the Inspector (screenshots above). Raw outputs are in raw/mcp_inspector_outputs.json.

## B. Domain MCP Server (s3209_trials)

Three tools over the s3209_rel data. All return the same envelope {ok, data, error}.

| Tool | Type | Input |
|---|---|---|
| search_trials | search | query (min 2 chars), status (optional), limit 1-25 |
| get_trial | detail lookup | trial_id (positive integer) |
| trial_stats | aggregate | group_by: status or sponsor |

**Code Snippet:**

```python
# ------------------------------------------------------------------ envelope
def ok(data: Any) -> dict:
    return {"ok": True, "data": data, "error": None}


def fail(message: str) -> dict:
    return {"ok": False, "data": None, "error": message}

```

Envelope {ok, data, error}

```python
    return f"unknown field(s): {', '.join(extra)}" if extra else None


def search_trials(store, inputs: dict) -> dict:
    if (msg := _bad_keys(inputs, "search_trials")):
        return fail(msg)
    query = inputs.get("query")
    if not isinstance(query, str) or len(query.strip()) < 2:
        return fail("query is required and must be a string of at least 2 characters")
    status = inputs.get("status")
    if status is not None and status not in TRIAL_STATUSES:
        return fail(f"status must be one of {list(TRIAL_STATUSES)}")
    limit = inputs.get("limit", 10)
    if isinstance(limit, bool) or not isinstance(limit, int) or not 1 <= limit <= MAX_LIMIT:
        return fail(f"limit must be an integer between 1 and {MAX_LIMIT}")
```

search_trials validation

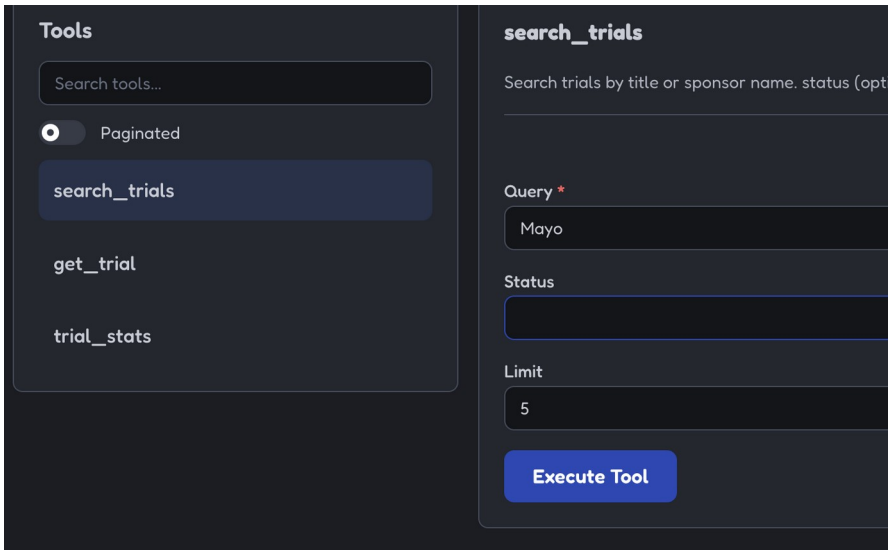
```python
def _run(name: str, inputs: dict) -> dict:
    # drop arguments the client left empty so defaults apply
    inputs = {k: v for k, v in inputs.items() if v is not None}
    log.info("tool=%s inputs=%s", name, inputs)
    try:
        result = core.TOOLS[name](store, inputs)
    except Exception as exc:  # DB down etc. -> clean envelope, never a crash
        log.exception("tool %s failed", name)
        result = core.fail(f"{type(exc).__name__}: {exc}")
    log.info("tool=%s ok=%s", name, result["ok"])
    return result


@mcp.tool()
def search_trials(query: str, status: str | None = None, limit: int = 10) -> dict:
    """Search trials by title or sponsor name. status (optional): Recruiting,
    Active - Not Recruiting, Completed, Terminated. limit: 1-25."""
    return _run("search_trials", {"query": query, "status": status, "limit": limit})

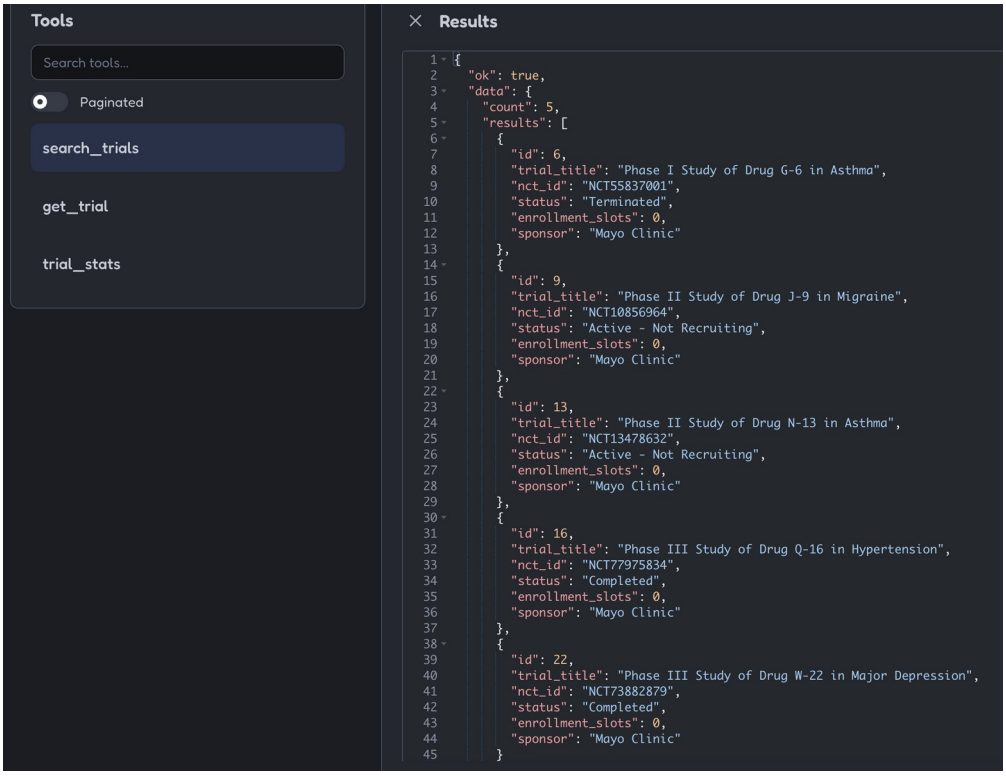
```

Server tool wrapper

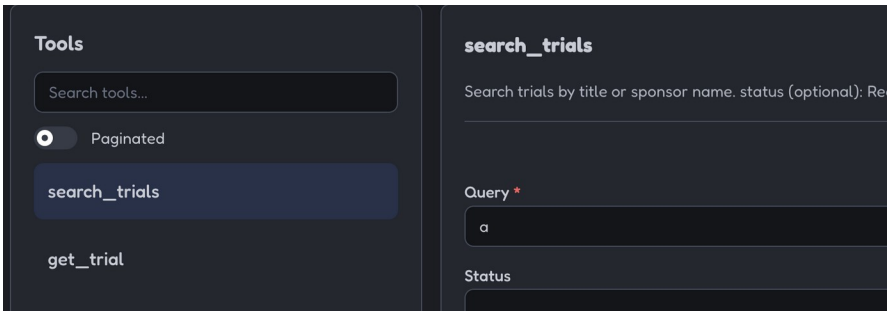
**Output Snippet (MCP Inspector):**



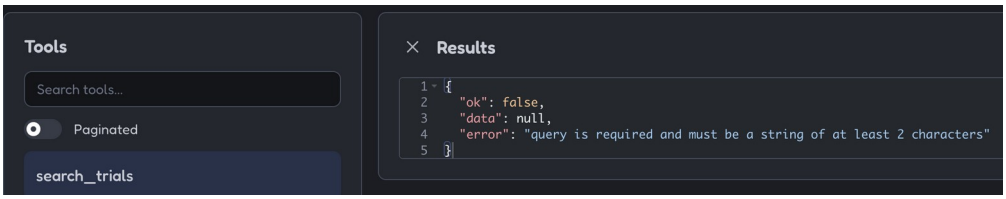

search_trials valid input



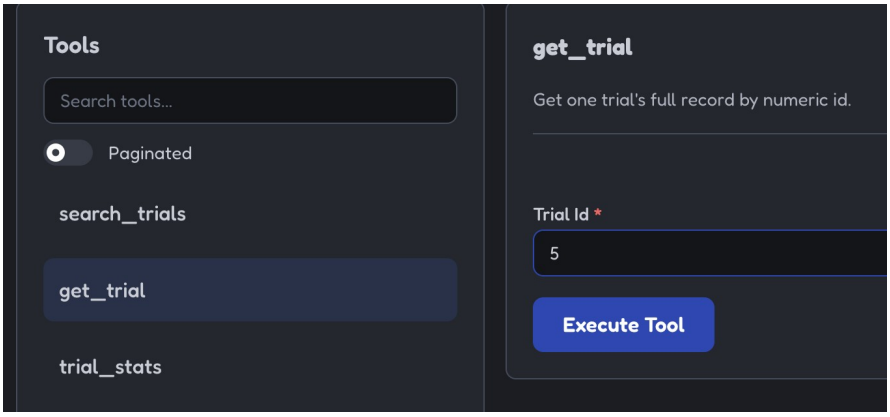

search_trials valid output



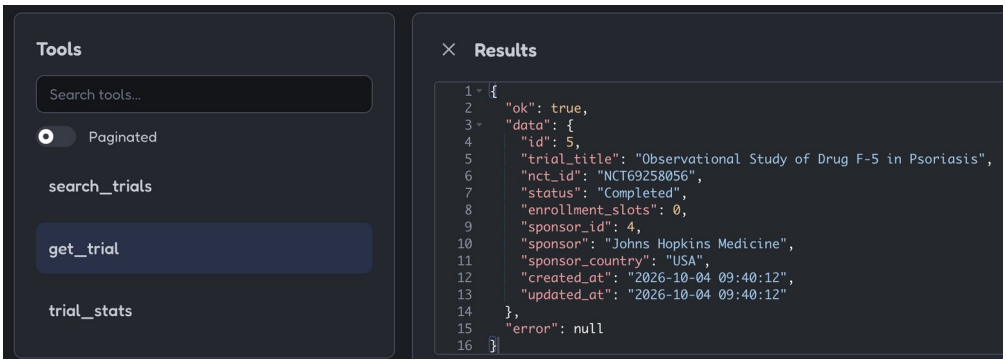

search_trials invalid input



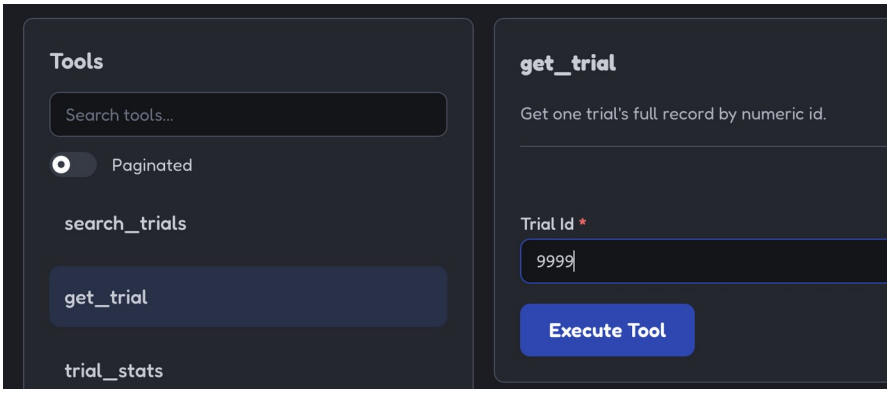

search_trials rejected



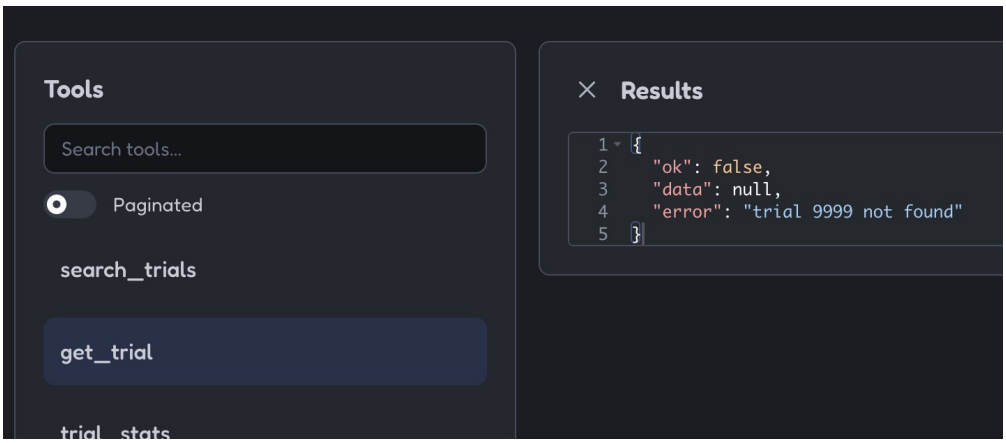

get_trial valid input



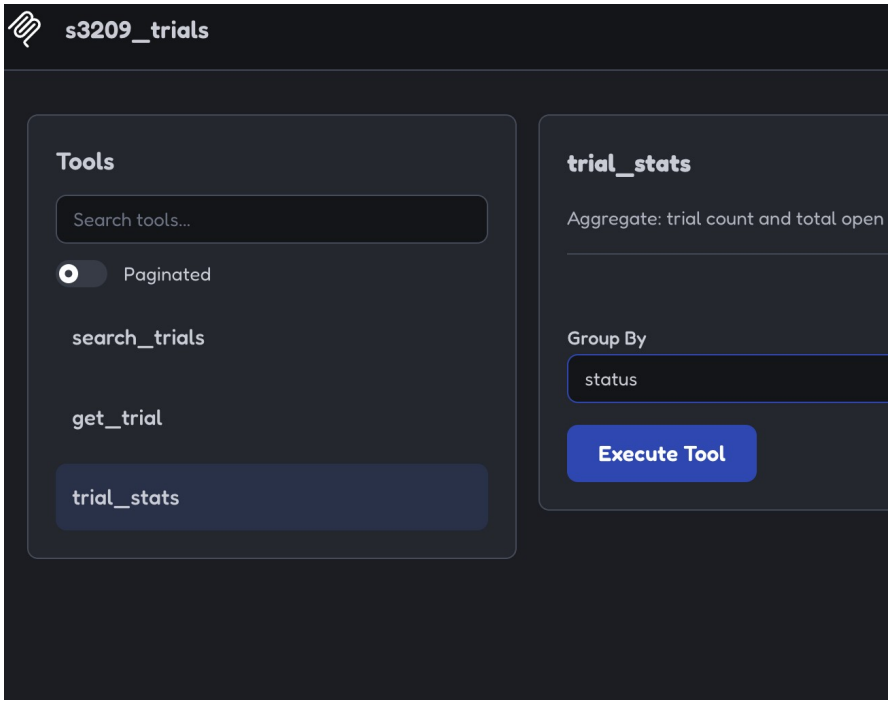

get_trial valid output



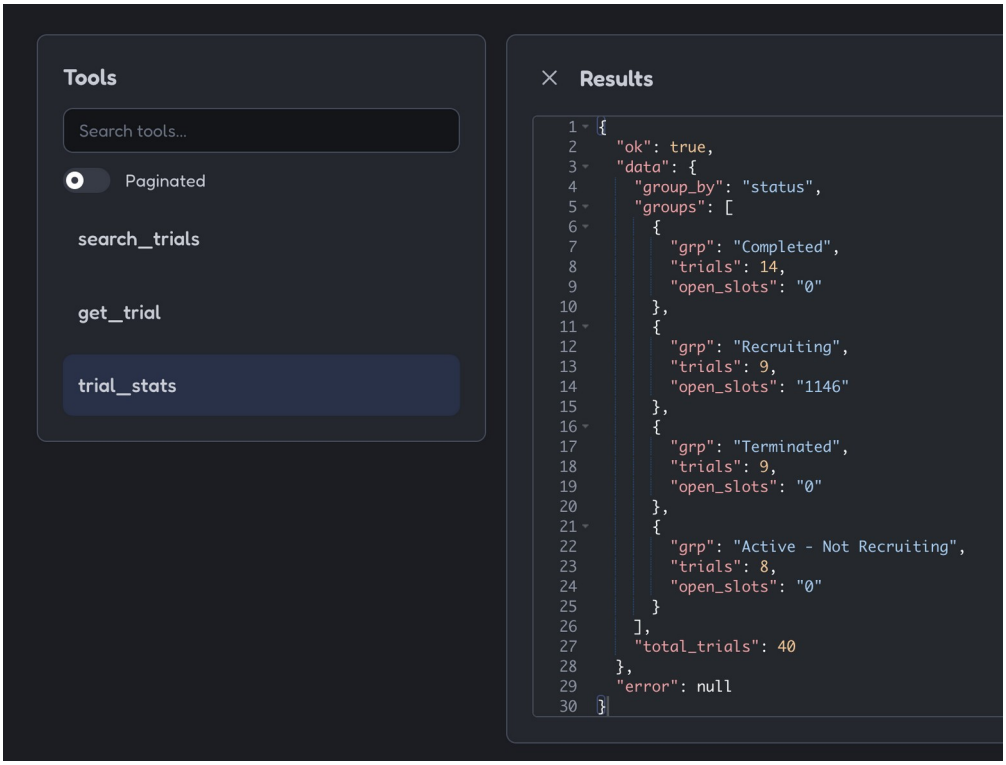

get_trial invalid input



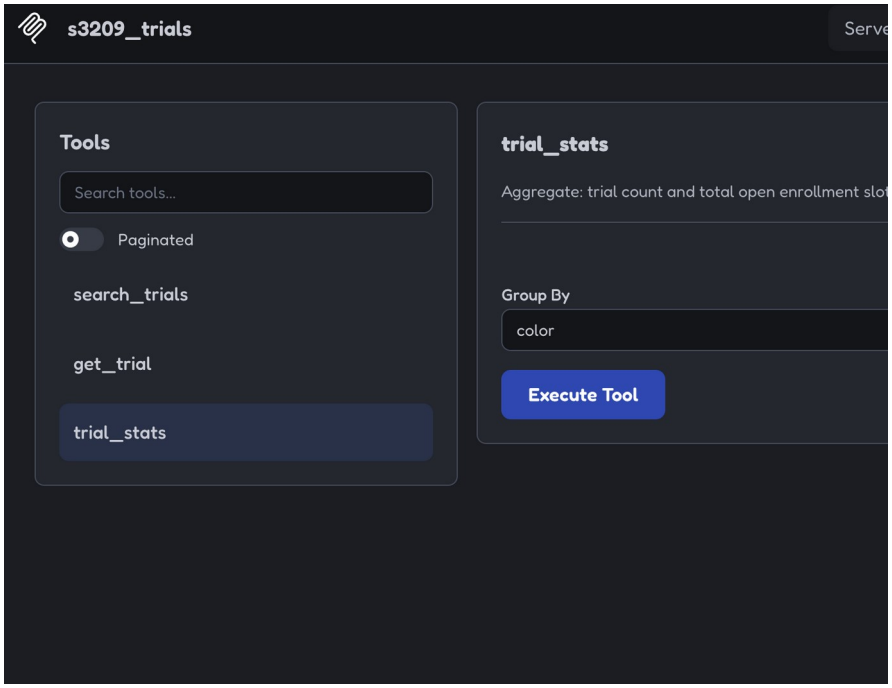
get_trial rejected




trial_stats valid input




trial_stats valid output




trial_stats invalid input




trial_stats rejected

# Part 3 - Tool Contracts Under Stress

## 18. Input schemas and rejected calls

### search_trials

```
Expected input schema:
{"type": "object",
 "properties": {
   "query":  {"type": "string", "minLength": 2},
   "status": {"type": "string",
              "enum": ["Recruiting", "Active - Not Recruiting",
                       "Completed", "Terminated"]},
   "limit":  {"type": "integer", "minimum": 1, "maximum": 25,
              "default": 10}},
```

```
  "required": ["query"], "additionalProperties": false}

 Rejected input:   {"query": "a"}
 Returned output:  {"ok": false, "data": null,
                    "error": "query is required and must be a
                              string of at least 2 characters"}
```

Why: the query has only 1 character and the minimum is 2.

**get_trial**

```
 Expected input schema:
 {"type": "object",
  "properties": {"trial_id": {"type": "integer", "minimum": 1}},
  "required": ["trial_id"], "additionalProperties": false}

 Rejected input:   {"trial_id": 9999}
 Returned output:  {"ok": false, "data": null, "error": "trial 9999 not found"}
```

Why: 9999 matches the schema but no trial with that id exists in s3209_rel.

**trial_stats**

```
 Expected input schema:
 {"type": "object",
  "properties": {"group_by": {"type": "string", "enum": ["status", "sponsor"],
                              "default": "status"}},
  "required": [], "additionalProperties": false}

 Rejected input:   {"group_by": "color"}
 Returned output:  {"ok": false, "data": null,
                    "error": "group_by must be one of ['status', 'sponsor']"}
```

Why: "color" is not in the allowed values (status or sponsor).

## 19. Timeouts and retries

Every storage call runs with a 1.0 s timeout and up to 3 attempts with exponential backoff (0.05 s, 0.10 s, max 0.4 s). If all attempts fail execute_tool returns {ok: false} instead of crashing.

**Code Snippet:**

```python
@dataclass
class RetryPolicy:
    max_attempts: int = 3        # 1 try + 2 retries
    timeout_s: float = 1.0       # per attempt
    base_delay_s: float = 0.05   # backoff: 0.05s, 0.10s, 0.20s ... (doubles)
    max_delay_s: float = 0.4     # never wait longer than this between attempts


_pool = ThreadPoolExecutor(max_workers=4)


def call_with_retry(fn: Callable[[], Any], policy: RetryPolicy = RetryPolicy(),
                    sleep: Callable[[float], None] = time.sleep,
                    on_attempt: Callable[[int, str], None] | None = None) -> tuple[Any, int]:
    """Run fn with a per-attempt timeout and bounded exponential backoff.

    Returns (result, attempts_used). Raises RetryExhausted when every attempt failed.
    """
    last = ""
    for attempt in range(1, policy.max_attempts + 1):
        try:
            result = _pool.submit(fn).result(timeout=policy.timeout_s)
            if on_attempt:
                on_attempt(attempt, "ok")
            return result, attempt
        except FutureTimeout:
            last = f"timeout after {policy.timeout_s}s"
        except (TransientError, ConnectionError, OSError) as exc:
            last = f"{type(exc).__name__}: {exc}"
        if on_attempt:
            on_attempt(attempt, last)
        if attempt < policy.max_attempts:
            sleep(min(policy.base_delay_s * 2 ** (attempt - 1), policy.max_delay_s))
    raise RetryExhausted(policy.max_attempts, last)
```

RetryPolicy and call_with_retry

**Output Snippet:**

```
python run_part3.py | tee -a ../../reports/hw05/RUN_LOG.txt
((.venv) ) anitaraiya@mac mcp_servers % cd ~/Desktop/data260-3209
cat reports/hw05/RUN_LOG.txt
[2026-10-04 10:05:04] ======================================================================
[2026-10-04 10:05:04] RUN: python run_part3.py   (HW5 Part 3, VERIFY_SEED=263209, data: s3209_rel)
[2026-10-04 10:05:04] ======================================================================
[2026-10-04 10:05:04] Retry policy: max_attempts=3, timeout=1.0s per attempt, backoff=0.05s x2^n (cap 0.4s)
[2026-10-04 10:05:04] ----------------------------------------------------------------------
[2026-10-04 10:05:04] A) success on first attempt
[2026-10-04 10:05:04]    attempt 1: ok
[2026-10-04 10:05:04]    -> success after 1 attempt(s): trial 5 'Observational Study of Drug F-5 in Psoriasis'
[2026-10-04 10:05:04] ----------------------------------------------------------------------
[2026-10-04 10:05:04] B) fail once, then succeed on retry
[2026-10-04 10:05:04]    attempt 1: TransientError: simulated dropped DB connection
[2026-10-04 10:05:04]    attempt 2: ok
[2026-10-04 10:05:04]    -> success after 2 attempt(s): trial 5 'Observational Study of Drug F-5 in Psoriasis'
[2026-10-04 10:05:04] ----------------------------------------------------------------------
[2026-10-04 10:05:04] C) fail on every attempt (timeouts)
[2026-10-04 10:05:05]    attempt 1: timeout after 1.0s
[2026-10-04 10:05:07]    attempt 2: timeout after 1.0s
[2026-10-04 10:05:08]    attempt 3: timeout after 1.0s
[2026-10-04 10:05:08]    -> RetryExhausted: storage unavailable after 3 attempts (last error: timeout after 1.0s)
[2026-10-04 10:05:11]    execute_tool returns a clean envelope instead of crashing:
   {"ok": false, "data": null, "error": "storage unavailable after 3 attempts (last error: timeout after 1.0s)"}
```

A success on first attempt, B success after retry, C fails after all retries

## 20. Fault injection with VERIFY_SEED

FaultyStore uses random.Random(263209), so the same seed always gives the same failures. 50 calls per rate (150 total) are saved in raw/fault_injection_calls.csv.

**Code Snippet:**

```python
class FaultyStore:
    """Wraps a store and fails each call with probability `failure_rate`.

    The random generator is seeded (VERIFY_SEED), so the same seed always gives
    the same success/failure sequence.
    """

    def __init__(self, store, failure_rate: float, seed: int = VERIFY_SEED,
                 latency_s: float = 0.002):
        self.store, self.rate, self.latency_s = store, failure_rate, latency_s
        self.rng = random.Random(seed)
        self.injected = 0

    def _maybe_fail(self):
        time.sleep(self.latency_s)                # simulated network latency
        if self.rng.random() < self.rate:
            self.injected += 1
            raise TransientError("injected fault")
```

FaultyStore with seeded random generator

**Results:**

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|---|---|---|---|
| 0% | 100% (50/50) | 3.34 | 6.47 |
| 20% | 100% (50/50) | 15.79 | 61.36 |
| 50% | 90% (45/50) | 54.31 | 169.52 |

```
[2026-10-04 10:05:14] ================================================================
[2026-10-04 10:05:14] Fault injection: 50 calls per rate, 150 calls total (raw data: reports/hw05/raw/fault_injection_calls.csv)
[2026-10-04 10:05:14] Injected failure rate   Success rate   Mean latency (ms)   p99 latency (ms)
[2026-10-04 10:05:14]   0%                    100.0%             3.34               6.47
[2026-10-04 10:05:14]  20%                    100.0%            15.79              61.36
[2026-10-04 10:05:14]  50%                     90.0%            54.31             169.52
((.venv) ) anitaraiya@mac data260-3209 %
```

Fault-injection results

### 21. Interactive vs. batch

**Interactive:** the policy is suitable. Even at 50% failures p99 is only 170 ms and the worst case (database hanging) gives up after about 3.2 s with a clear error. For a chat I could lower the timeout to 0.5 s.

**Batch:** time matters less than finishing every record. I would use 6-8 attempts, a 5-10 s timeout and backoff starting at 1 s capped at 30-60 s with jitter. With 8 attempts success at 50% failures is about 99.6%. Records that still fail go to a retry list.

## Part 4 - One Safe Tool Entry Point and Offline Tests

### 22. execute_tool(name, inputs)

Single entry point for the three tools. It returns the {ok, data, error} envelope as a JSON string and never crashes.

**Code Snippet:**

```python
def execute_tool(name: str, inputs: dict | None = None, store=None) -> str:
    """Validate, apply the safety rule, run the tool, and return the envelope as JSON."""
    try:
        if name not in core.TOOLS:
            result = core.fail(f"unknown tool '{name}'; available: {sorted(core.TOOLS)}")
        elif inputs is not None and not isinstance(inputs, dict):
            result = core.fail("inputs must be a JSON object")
        elif (reason := safety_violation(inputs or {})):
            result = core.fail(reason)
        else:
            result = core.TOOLS[name](store or default_store(), dict(inputs or {}))
    except RetryExhausted as exc:
        result = core.fail(str(exc))
    except Exception as exc:  # last line of defence: never crash the agent
        result = core.fail(f"{type(exc).__name__}: {exc}")
    return json.dumps(result, default=str)

```

execute_tool

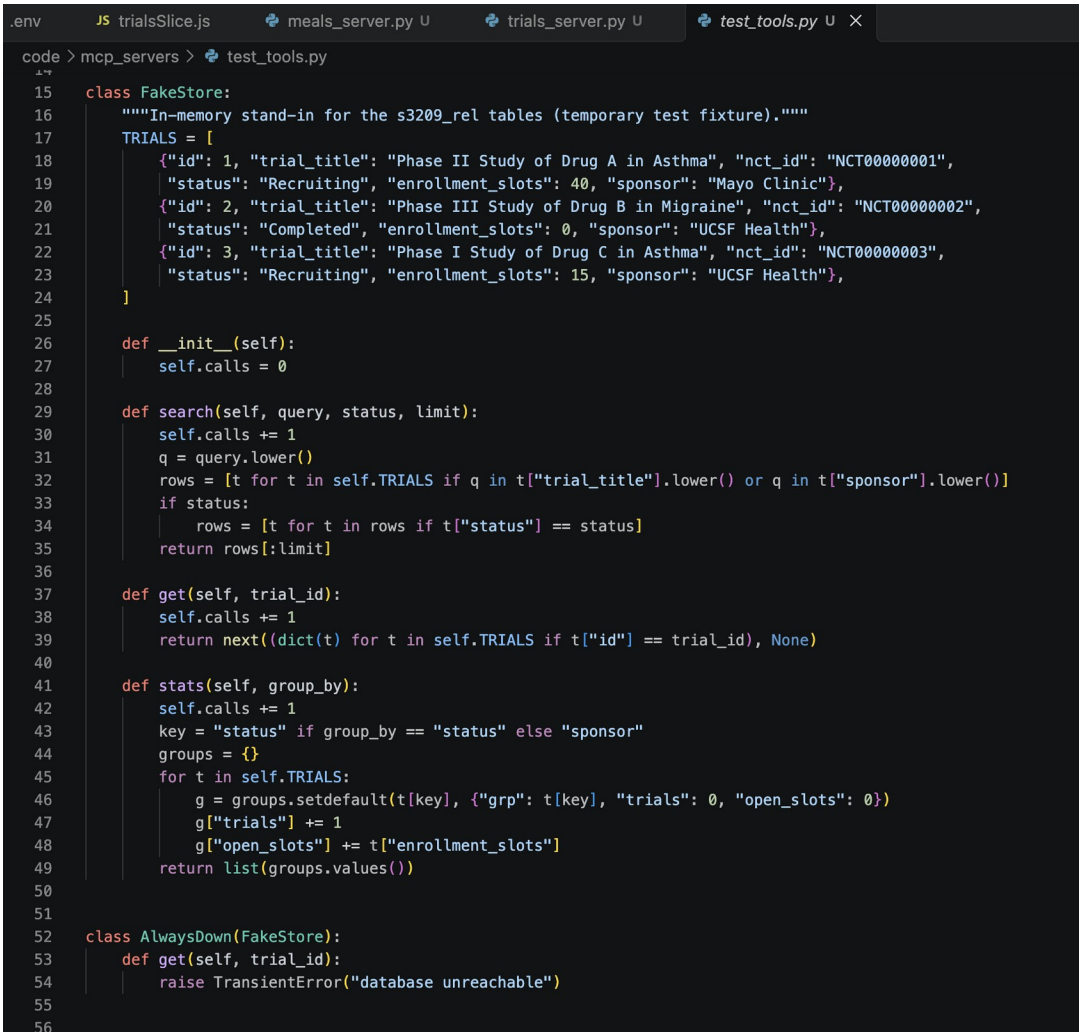
### 23. Test runner

test_tools.py uses plain assert statements, prints PASS or FAIL for each test and an X/Y summary. It tests a valid input and the Part 3 invalid input for each tool plus an unknown tool and a database outage.

### 24. Offline fixtures

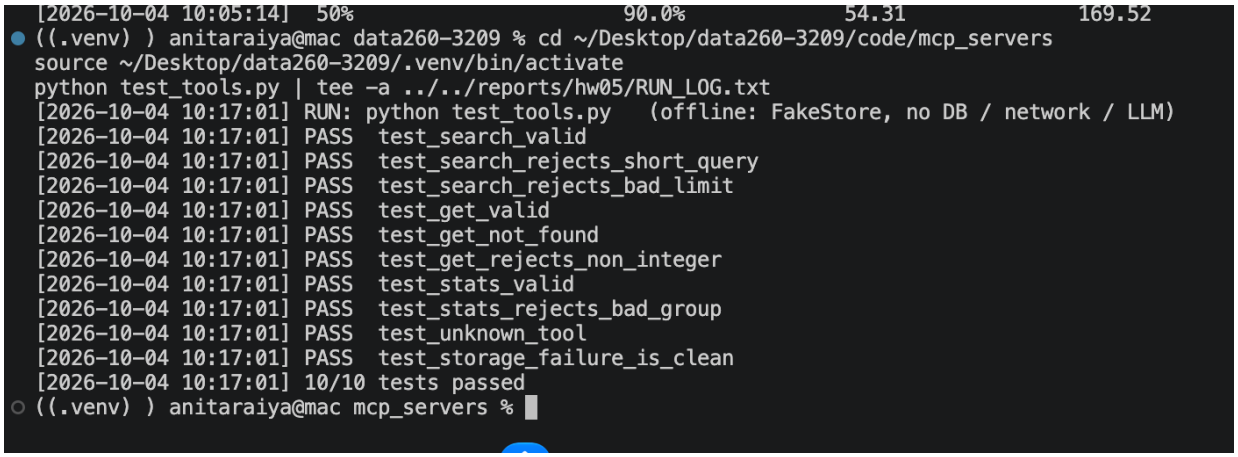
A FakeStore is passed into execute_tool, so the tests run offline without a database, network or LLM.

**Code Snippet:**




FakeStore and AlwaysDown fixtures

**Output Snippet:**




Test runner - 10/10 passed

# Part 5 - Agent Loop, Safety Rule and Reflection

## I. Safety rule

Tool inputs with patient-identifying data (email, SSN, phone number or MRN) are blocked inside execute_tool with {ok: false, data: null, error: "blocked by safety rule ..."} and no exception.

### Code Snippet:

```python
# Domain rule: a public trial registry must never receive patient-identifying data.
# Tool inputs that contain an email address, phone number, SSN or medical record
# number (MRN) are blocked before any database call is made.
PHI_PATTERNS = {
    "email address": re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"),
    "SSN": re.compile(r"\b\d{3}-\d{2}-\d{4}\b"),
    "phone number": re.compile(r"\b\d{3}[-. ]\d{3}[-. ]\d{4}\b"),
    "medical record number": re.compile(r"\bMRN\s*[:#]?\s*\d+", re.IGNORECASE),
}


def safety_violation(inputs: dict) -> str | None:
    for value in inputs.values():
        if isinstance(value, str):
            for label, pattern in PHI_PATTERNS.items():
                if pattern.search(value):
                    return (f"blocked by safety rule: input contains patient-identifying "
                            f"data ({label}); the trial registry only accepts de-identified queries")
    return None

```

Safety rule

### Output Snippet:

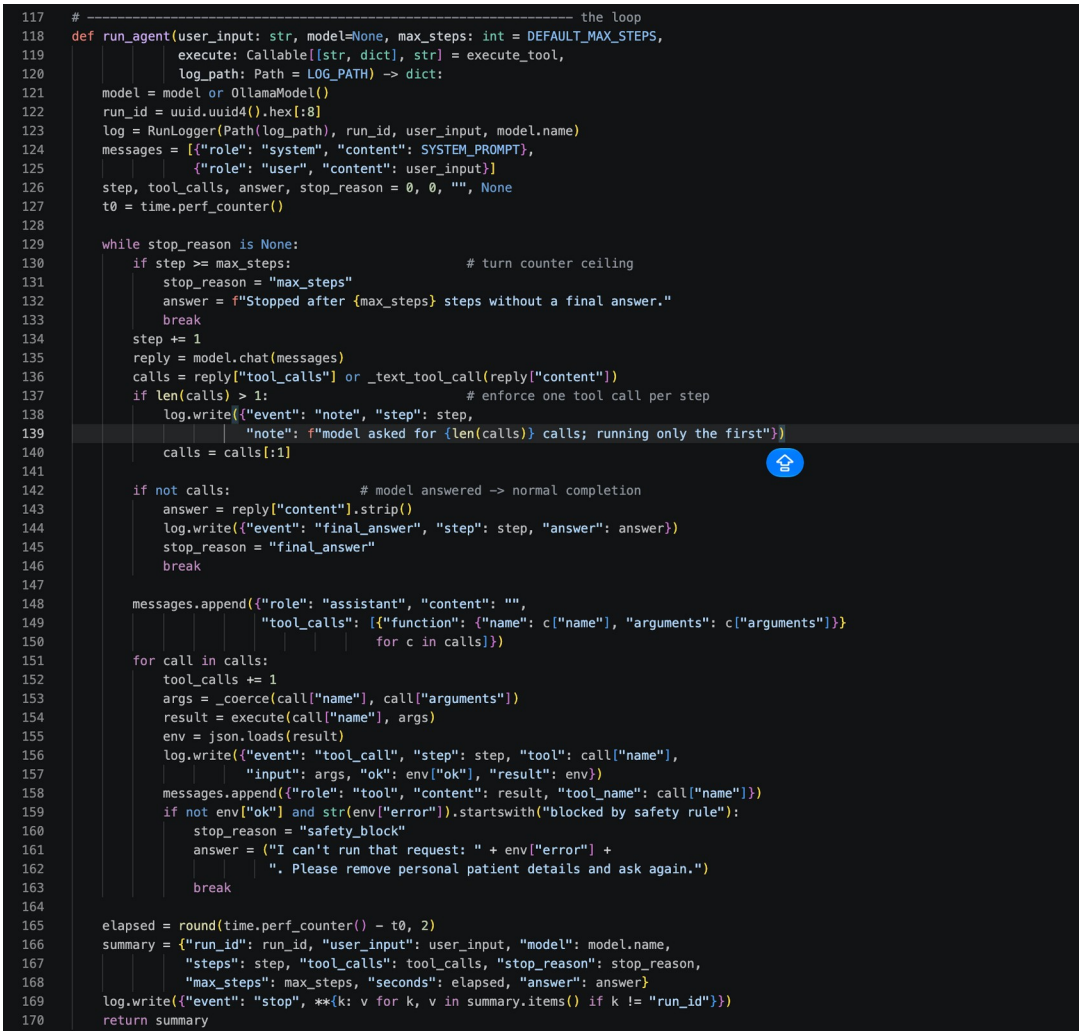
```
[2026-10-04 10:21:11] step log: reports/hw05/raw/agent_runs.jsonl  summary: reports/hw05/raw/agent_scenarios.json
((.venv) ) anitaraiya@mac mcp_servers % cd ~/Desktop/data260-3209/code/mcp_servers
source ~/Desktop/data260-3209/.venv/bin/activate
python run_part5.py | tee -a ../../reports/hw05/RUN_LOG.txt
[2026-10-04 10:26:22] ============================================================
[2026-10-04 10:26:22] RUN: python run_part5.py   (HW5 Part 5)
[2026-10-04 10:26:22] I. Safety rule inside execute_tool: no patient-identifying data in tool inputs
[2026-10-04 10:26:23]   allowed call  search_trials {'query': 'asthma', 'limit': 3}
[2026-10-04 10:26:23]     -> ok=True, 3 trials returned
[2026-10-04 10:26:23]   blocked call  search_trials {'query': 'asthma trial for john.doe@gmail.com'}
[2026-10-04 10:26:23]     -> {"ok": false, "data": null, "error": "blocked by safety rule: input contains patient-identifying data (email address); the trial registry only accepts de-identifi
ed queries"}
```

Allowed call and blocked call

## II. Agent loop

run_agent uses llama3.1:8b through Ollama with the three tools via execute_tool. It has a step counter and max_steps and logs every step to raw/agent_runs.jsonl. Stop reasons: final_answer, safety_block or max_steps.

**Code Snippet:**



```python
# ------------------------------------------------------------------ the loop
def run_agent(user_input: str, model=None, max_steps: int = DEFAULT_MAX_STEPS,
              execute: Callable[[str, dict], str] = execute_tool,
              log_path: Path = LOG_PATH) -> dict:
    model = model or OllamaModel()
    run_id = uuid.uuid4().hex[:8]
    log = RunLogger(Path(log_path), run_id, user_input, model.name)
    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_input}]
    step, tool_calls, answer, stop_reason = 0, 0, "", None
    t0 = time.perf_counter()

    while stop_reason is None:
        if step >= max_steps:                       # turn counter ceiling
            stop_reason = "max_steps"
            answer = f"Stopped after {max_steps} steps without a final answer."
            break
        step += 1
        reply = model.chat(messages)
        calls = reply["tool_calls"] or _text_tool_call(reply["content"])
        if len(calls) > 1:                          # enforce one tool call per step
            log.write({"event": "note", "step": step,
                       "note": f"model asked for {len(calls)} calls; running only the first"})
            calls = calls[:1]

        if not calls:                # model answered -> normal completion
            answer = reply["content"].strip()
            log.write({"event": "final_answer", "step": step, "answer": answer})
            stop_reason = "final_answer"
            break

        messages.append({"role": "assistant", "content": "",
                         "tool_calls": [{"function": {"name": c["name"], "arguments": c["arguments"]}}
                                        for c in calls]})
        for call in calls:
            tool_calls += 1
            args = _coerce(call["name"], call["arguments"])
            result = execute(call["name"], args)
            env = json.loads(result)
            log.write({"event": "tool_call", "step": step, "tool": call["name"],
                       "input": args, "ok": env["ok"], "result": env})
            messages.append({"role": "tool", "content": result, "tool_name": call["name"]})
            if not env["ok"] and str(env["error"]).startswith("blocked by safety rule"):
                stop_reason = "safety_block"
                answer = ("I can't run that request: " + env["error"] +
                          ". Please remove personal patient details and ask again.")
                break

    elapsed = round(time.perf_counter() - t0, 2)
    summary = {"run_id": run_id, "user_input": user_input, "model": model.name,
               "steps": step, "tool_calls": tool_calls, "stop_reason": stop_reason,
               "max_steps": max_steps, "seconds": elapsed, "answer": answer}
    log.write({"event": "stop", **{k: v for k, v in summary.items() if k != "run_id"}})
    return summary
```

run_agent loop

## III. Additional tests

Added two offline tests: the safety rule blocks a call and run_agent with MockModel stops at max_steps.

**Code Snippet:**

```python
# HW3 Part 3 III - two extra offline tests (added to the Part 4 runner in test_tools.py):
import json
import tempfile
from pathlib import Path

from agent import MockModel, run_agent
from tool_gateway import execute_tool


def test_safety_rule_blocks_patient_data():
    from test_tools import FakeStore
    store = FakeStore()
    env = json.loads(execute_tool("search_trials",
                                  {"query": "asthma trial for john.doe@gmail.com"}, store=store))
    assert env["ok"] is False and env["data"] is None
    assert env["error"].startswith("blocked by safety rule")
    assert store.calls == 0, "blocked call must never reach the data store"


def test_agent_stops_at_max_steps():
    from test_tools import FakeStore
    always_tool = MockModel([{"content": "", "tool_calls": [
        {"name": "get_trial", "arguments": {"trial_id": 1}}]}])
    log = Path(tempfile.mkdtemp()) / "agent_runs.jsonl"
    out = run_agent("loop forever", model=always_tool, max_steps=3,
                    execute=lambda n, i: execute_tool(n, i, store=FakeStore()), log_path=log)
    assert out["stop_reason"] == "max_steps"
    assert out["steps"] == 3 and out["tool_calls"] == 3
    events = [json.loads(line)["event"] for line in log.read_text().splitlines()]
    assert events == ["start", "tool_call", "tool_call", "tool_call", "stop"]


TESTS = [test_safety_rule_blocks_patient_data, test_agent_stops_at_max_steps]

```

Two new tests

```python
class MockModel:
    """Offline stand-in for tests: replays scripted replies (the last one repeats forever)."""

    def __init__(self, replies: list[dict]):
        self.replies, self.i, self.name = replies, 0, "MockModel"

    def chat(self, messages: list[dict]) -> dict:
        reply = self.replies[min(self.i, len(self.replies) - 1)]
        self.i += 1
        return reply
```

MockModel

**Output Snippet:**

```
((.venv) ) anitaraiya@mac mcp_servers % cd ~/Desktop/data260-3209/code/mcp_servers
source ~/Desktop/data260-3209/.venv/bin/activate
pip install ollama
python test_tools.py | tee -a ../../reports/hw05/RUN_LOG.txt
Requirement already satisfied: ollama in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site-packag
Requirement already satisfied: httpx>=0.27 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site-p
Requirement already satisfied: pydantic>=2.9 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site
Requirement already satisfied: anyio in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site-package
Requirement already satisfied: certifi in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site-packa
Requirement already satisfied: httpcore==1.* in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site
Requirement already satisfied: idna in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site-packages
Requirement already satisfied: h11>=0.16 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3.12/site-pac
Requirement already satisfied: annotated-types>=0.6.0 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python
Requirement already satisfied: pydantic-core==2.46.5 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/python3
Requirement already satisfied: typing-extensions>=4.14.1 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/pyt
Requirement already satisfied: typing-inspection>=0.4.2 in /Users/anitaraiya/Desktop/data260-3209/.venv/lib/pyth
[2026-10-04 10:20:33] RUN: python test_tools.py   (offline: FakeStore, no DB / network / LLM)
[2026-10-04 10:20:33] PASS  test_search_valid
[2026-10-04 10:20:33] PASS  test_search_rejects_short_query
[2026-10-04 10:20:33] PASS  test_search_rejects_bad_limit
[2026-10-04 10:20:33] PASS  test_get_valid
[2026-10-04 10:20:33] PASS  test_get_not_found
[2026-10-04 10:20:33] PASS  test_get_rejects_non_integer
[2026-10-04 10:20:33] PASS  test_stats_valid
[2026-10-04 10:20:33] PASS  test_stats_rejects_bad_group
[2026-10-04 10:20:33] PASS  test_unknown_tool
[2026-10-04 10:20:33] PASS  test_storage_failure_is_clean
[2026-10-04 10:20:33] PASS  test_safety_rule_blocks_patient_data
[2026-10-04 10:20:33] PASS  test_agent_stops_at_max_steps
[2026-10-04 10:20:33] 12/12 tests passed
((.venv) ) anitaraiya@mac mcp_servers %
```

Test runner - 12/12 passed

## IV. Metrics

| Scenario | max_steps | Step count | Tool-call count | Stop reason |
|---|---|---|---|---|
| S1 Recruiting trials and open slots | 6 | 2 | 1 | final_answer |
| S2 Mayo Clinic first trial details | 6 | 6 | 5 | final_answer |
| S3 Query with MRN 448812 | 6 | 1 | 1 | safety_block |
| S4 Fetch trials 1-5 before answering | 3 | 3 | 3 | max_steps |

```
[2026-10-04 10:26:23] ------------------------------------------------------------
[2026-10-04 10:26:23] II–IV. Agent scenarios with local model llama3.1:8b (temperature 0, seed 3209)
[2026-10-04 10:26:25] ------------------------------------------------------------
[2026-10-04 10:26:25] S1 aggregate: How many trials are currently Recruiting, and how many open enrollment slots do they have in total?
[2026-10-04 10:26:25]    run_id=505c2784 steps=2 tool_calls=1 stop_reason=final_answer time=2.29s
[2026-10-04 10:26:25]    answer: Based on the tool's output, there are 9 trials that are currently Recruiting, and they have a total of 1146 open enrollment slots.
[2026-10-04 10:26:31] ------------------------------------------------------------
[2026-10-04 10:26:31] S2 search + detail: Find trials sponsored by Mayo Clinic, then give me the full details of the first trial in the results.
[2026-10-04 10:26:31]    run_id=548a7b47 steps=6 tool_calls=5 stop_reason=final_answer time=6.23s
[2026-10-04 10:26:31]    answer: The first trial sponsored by Mayo Clinic is "Phase I Study of Drug G-6 in Asthma" with the ID NCT55837001. It is a terminated trial with 0 enrollment slots.
[2026-10-04 10:26:32] ------------------------------------------------------------
[2026-10-04 10:26:32] S3 safety rule: Search the registry using exactly this query text, unchanged: 'asthma patient MRN 448812'
[2026-10-04 10:26:32]    run_id=1621e26a steps=1 tool_calls=1 stop_reason=safety_block time=0.64s
[2026-10-04 10:26:32]    answer: I can't run that request: blocked by safety rule: input contains patient-identifying data (medical record number); the trial registry only accepts de-identifie
d queries. Please remove personal patient details and ask again.
[2026-10-04 10:26:33] ------------------------------------------------------------
[2026-10-04 10:26:33] S4 max_steps: Get the full record of trial 1, then trial 2, then trial 3, then trial 4, then trial 5, one tool call at a time, and only answer after all five.
[2026-10-04 10:26:33]    run_id=aefd6034 steps=3 tool_calls=3 stop_reason=max_steps time=1.41s
[2026-10-04 10:26:33]    answer: Stopped after 3 steps without a final answer.
[2026-10-04 10:26:33] ============================================================
[2026-10-04 10:26:33] Scenario           Steps   Tool calls  Stop reason
[2026-10-04 10:26:33] S1 aggregate       2       1           final_answer
[2026-10-04 10:26:33] S2 search + detail 6       5           final_answer
[2026-10-04 10:26:33] S3 safety rule     1       1           safety_block
[2026-10-04 10:26:33] S4 max_steps       3       3           max_steps
[2026-10-04 10:26:33] step log: reports/hw05/raw/agent_runs.jsonl   summary: reports/hw05/raw/agent_scenarios.json
((.venv) ) anitaraiya@mac mcp_servers %
```

Agent scenarios

### V. Reflection

Run 548a7b47 (S2): "Find trials sponsored by Mayo Clinic, then give me the full details of the first trial in the results." Model llama3.1:8b, max_steps = 6.

Step 1: the model asked for two calls at once. The harness ran only the first one, search_trials {query: "Mayo Clinic", limit: 10}. It passed the safety rule and returned Mayo Clinic trials with trial 6 first.

Steps 2-4: the model called get_trial with wrong argument names ({id: "6"}, then {nct_id: "NCT55837001"}, then {id: "6"} again). Each time execute_tool returned {ok: false, error: "unknown field(s): ..."} instead of crashing. The harness logged the step, sent the error back to the model and increased the step counter.

Step 5: the model corrected the call to get_trial {trial_id: 6} and got ok: true with the full record.

Step 6: the model answered with no tool call: trial 6 is "Phase I Study of Drug G-6 in Asthma" (NCT55837001), Terminated, 0 open slots. This matches the database.

Why it stopped: normal completion (final_answer) at step 6, exactly at the max_steps limit. One more bad call and it would have stopped with max_steps. The envelope turned rejected calls into feedback instead of errors. It also showed that a small local model wastes steps on argument names, so error messages should name the correct field.

## AI Use

**1. What I used an AI assistant for and what I did myself.** I used an AI assistant to help write the code. It also helped me move the project from Windows to Mac and helped to in writing like checking grammatical error and framing sentence. I did the setup and running myself. I ran MySQL, the backend, the React app, Postman, the MCP Inspector and Ollama, took all the screenshots and checked every output against the database. It also helped me brainstorm and learn new concepts that I was not familiar with.

**2. One AI-produced output that was wrong.** The first version of the agent loop let llama3.1:8b send "status": "null" as text and ask for two tool calls at once. In scenario S2 it guessed get_trial(1) before the search finished and answered that trial 1 was a Mayo Clinic trial.

**3. How I found it.** I read agent_runs.jsonl and compared the answer with the database: trial 1 belongs to Charite Berlin. The log also showed that the search had been rejected because of the "null" status and that S3 never reached the safety rule.

**4. What I changed and why it works now.** The agent now runs only one tool call per step, drops empty or null optional arguments and reads tool calls written as text. I re-ran the 4 scenarios: S2 searched first and returned the correct trial (6, NCT55837001), S3 stopped with safety_block and the 12 offline tests still pass.